

Leveraging the Defects Life Cycle to Label Affected Versions and Defective Classes

BAILEY VANDEHEI, California Polytechnic State University
DANIEL ALENCAR DA COSTA, University of Otago, New Zealand
DAVIDE FALESSI, University of Rome Tor Vergata, Italy

Two recent studies explicitly recommend labeling defective classes in releases using the affected versions (AV) available in issue trackers (e.g., Jira). This practice is coined as the *realistic approach*. However, no study has investigated whether it is feasible to rely on AVs. For example, how available and consistent is the AV information on existing issue trackers? Additionally, no study has attempted to retrieve AVs when they are unavailable. The aim of our study is threefold: (1) to measure the proportion of defects for which the realistic method is usable, (2) to propose a method for retrieving the AVs of a defect, thus making the realistic approach usable when AVs are unavailable, (3) to compare the accuracy of the proposed method versus three SZZ implementations. The assumption of our proposed method is that defects have a stable life cycle in terms of the proportion of the number of versions affected by the defects before discovering and fixing these defects. Results related to 212 open-source projects from the Apache ecosystem, featuring a total of about 125,000 defects, reveal that the realistic method cannot be used in the majority (51%) of defects. Therefore, it is important to develop automated methods to retrieve AVs. Results related to 76 open-source projects from the Apache ecosystem, featuring a total of about 6,250,000 classes, affected by 60,000 defects, and spread over 4,000 versions and 760,000 commits, reveal that the proportion of the number of versions between defect discovery and fix is pretty stable (standard deviation <2)—across the defects of the same project. Moreover, the proposed method resulted significantly more accurate than all three SZZ implementations in (i) retrieving AVs, (ii) labeling classes as defective, and (iii) in developing defects repositories to perform feature selection. Thus, when the realistic method is unusable, the proposed method is a valid automated alternative to SZZ for retrieving the origin of a defect. Finally, given the low accuracy of SZZ, researchers should consider re-executing the studies that have used SZZ as an oracle and, in general, should prefer selecting projects with a high proportion of available and consistent AVs.

CCS Concepts: • **Software and its engineering → Software verification and validation**; **Software testing and debugging**;

Additional Key Words and Phrases: Affected version, SZZ, defect origin, developing defects repository

ACM Reference format:
Bailey Vandehei, Daniel Alencar Da Costa, and Davide Falessi. 2021. Leveraging the Defects Life Cycle to Label Affected Versions and Defective Classes. *ACM Trans. Softw. Eng. Methodol.* 30, 2, Article 24 (February 2021), 35 pages.
https://doi.org/10.1145/3433928

Authors' addresses: B. Vandehei, California Polytechnic State University, San Luis Obispo, 3940-7 Broad st #257 San Luis Obispo, CA 93401-0000, USA; email: bvandehe@calpoly.edu; D. A. da Costa, University of Otago, Dunedin, 60 Clyde Street, Dunedin, New Zealand, 09016; email: danielcalencar@otago.ac.nz; D. Falessi, University of Rome Tor Vergata, Via del Politecnico, 1 00133 – Roma, Italy; email: falessi@ing.uniroma2.it.
Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
© 2021 Association for Computing Machinery.
1049-331X/2021/02-ART24 $15.00
https://doi.org/10.1145/3433928

1 INTRODUCTION

The manner in which defects are introduced into code, and the sheer volume of defects in software, are typically beyond the capability and resources of most development teams [25, 36, 42, 51, 72, 74]. Due to this problem, researchers have explored machine learning approaches to predict (1) whether a defect is likely to occur in a software module (i.e, a binary response); and (2) how many post-deployment defects are likely to occur (a count response) [7, 21, 22, 32, 37, 44, 51, 52, 64, 68].

Defect prediction models are built using complexity features (i.e., lines of code, Halstead metrics, McCabe's complexity, and CK metrics) [3, 28, 46] and process features [56]. Predicting the occurrence of defects is useful because a development team could better focus the limited testing effort.

Before any defect prediction can be performed, it is important to create a repository containing the features and the associated defects. Our work focuses on the automated methods for the creation of defect prediction datasets. We are interested in methods for establishing the origin of a defect. Researchers provided means to create [24, 48, 87], collect [16], and select [26, 50, 63] datasets for associating software defects to process and product features. However, existing research has shown that the general quality of software defect datasets are not without flaws [5, 30, 38, 58, 71]. For example, Bird et al. [5] demonstrated the existence of a non-negligible bias in the features that are used to build defect prediction models. Tantithamthavorn et al. [71] have also shown that cleaning the datasets prior to performing defect predictions can increase the ability to better identify the defective modules. Indeed, the general accuracy of a defect prediction model depends on the quality of the underlying datasets [41, 66].

One main limitation of defect prediction models is the granularity of the predictions (e.g., whether a module is defective or not), which is often too coarse to be useful [35]. To face this limitation, researchers have explored Just-In-Time (JIT) defect predictions [20], in which the goal of the prediction is to indicate whether a newly produced commit will be defective or clean. Nevertheless, JIT prediction models can only be feasible if the exact origins of a defect are known [20].

To identify the origins of a defect, researchers have proposed the SZZ approach [69]. However, the state-of-the-art of the SZZ approach is far from being ideal [10, 60, 62]. For example, Costa et al. [10] highlighted that current SZZ implementations cannot determine the origins of defects that were fixed by solely adding code. Additionally, SZZ is also incapable of identifying the origins of defects of the regression type [75]. Finally, Rodríguez-Pérez et al. [61] revealed that only a significant minority of defects can have their origins traceable in the source code repository, thus limiting the applicability of SZZ.

Two recent studies [10, 83] suggest the use of affected versions (AVs) available in defect reports—which can be provided in issue trackers such as Jira—to better label defective modules, instead of solely relying on SZZ. However, these studies also hint that the availability of AVs is scarce [10, 83], i.e., only a few defect reports provide AVs. In this work, we propose a first-of-its-kind method to retrieve AV. The method, if used in combination with fix commit information, is used also to label defective classes. To achieve our goal, we first investigate the extent to which AVs are usable, i.e., available and consistent, in open-source projects. Second, we propose, evaluate, and compare novel and automated methods for retrieving AVs, including the earliest possible AV (i.e., the origin of the defect). Our intuition is that *defects have a stable life cycle* in terms of the proportion of the number of versions required to discover and to fix a defect. The idea is that defects that quickly manifest themselves as a fault (i.e., report creation) are easiest to find and fix than defects that are dormant over several releases [1, 8]. This is because developers need to identify the change that induced the defect to fix the defect. Our assumption is that: the older the defect-inducing change is, the longer it takes for it to be identified and fixed. The assumption of the stability of defects' life-cycle seem to have analogies with *diseases'* life-cycle [59].

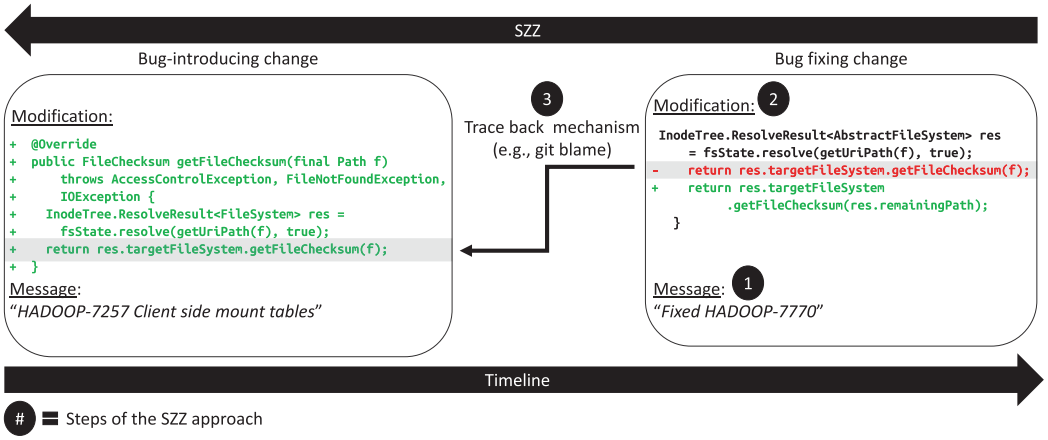


Fig. 1. An example of the SZZ approach. Step 1 consists of identifying the defect-fixing changes. Step 2 localizes the faulty code, which is the code removed in the defect-fixing change. Finally, in Step 3, SZZ traces the code history to find the defect-introducing changes.

Our results obtained in 212 Apache open-source projects reveal that AV information is lacking in the majority of defects (51%). Therefore, it is important to investigate automated methods for retrieving AVs. Our results obtained in 76 Apache open-source projects demonstrate that our proposed method is more accurate than previously proposed SZZ methods in terms of retrieving AVs. Additionally, our methods are more accurate in labeling classes as defective and in developing defects datasets for performing feature selection.

The remainder of this article is structured as follows. We explain the background material and related work in Section 2. In Section 3, we describe our study design. We present our obtained results in Section 4. In Section 5, we discuss our results. We explain the threats to validity of this study in Section 6, while we provide our conclusions in Section 7.

2 RELATED WORK & BACKGROUND

We provide the key concepts to understand our research context in this section.

Śliwerski et al. [69] proposed the first implementation of the SZZ approach, which strove to find the origins of a defect (i.e., the defect-introducing changes). SZZ exploits the versioning system annotation mechanism (e.g., git blame) to determine—for the source code lines that have been changed in a defect fix—when they have last been changed before the fix. The SZZ approach consists of three main steps. We demonstrate these steps by using the HADOOP-7770[1] defect as an example (shown in Figure 1). HADOOP-7770 was caused because the developers used the wrong object to provide a file path, which incurred a `FileNotFoundException`. **Step 1** of SZZ (shown in Figure 1) consists of finding the change that fixed the defect (i.e., the defect-fixing change). In the case of HADOOP-7770, the defect-fixing change was performed in change `1190532`[2] by changing `getFileChecksum(f)` to `getFileChecksum(res.remainingPath)`. SZZ can use several mechanisms to find defect-fixing changes [69].[3] Afterward, in **Step 2**, SZZ analyzes the diff patch of the

[1] https://issues.apache.org/jira/browse/HADOOP-7770.
[2] http://svn.apache.org/viewvc?view=revision&revision=1190541.
[3] A popular approach to identify defect-fixing changes is to use simple heuristics, such as searching for the "fix" or "fixed" keywords in a change log [14]. However, the SZZ implementations used in this work search for defect IDs within change logs for identifying the defect-fixing changes.

defect-fixing change to locate the *faulty code*. In this step, SZZ assumes that the code removed in a patch is the code that expresses the defect. In the case of HADOOP-7770, the removed code in the diff patch was the `getFileChecksum(f);` code. Finally, once the *faulty code* has been identified, SZZ traces the code history to find when the *faulty code* was introduced (i.e., **Step 3**). **Step 3** of SZZ can be implemented by using, for example, the `blame` operation that is present in most Version Control Systems (VCSs, such as Git or Subversion). In Figure 1, SZZ uses the *git blame* command to find the change, `1100026`[4], which is the change that introduced the `getFileChecksum(f);` and, hence, the code that potentially introduced the defect (i.e., the defect-introducing change).

Several other studies strove to estimate the origin of defects. Kim et al. [40] presented algorithms to automatically and accurately identify defect-introducing changes, which improved over SZZ. Costa et al. [10] proposed three criteria and evaluated five SZZ implementations. They concluded that current SZZ implementations still lack mechanisms to accurately identify defect-introducing changes. Yatish et al. [83] presented the realistic approach (i.e., using AVs) to estimate the origin of a defect. This approach relies on the use of the AV and is the main motivation of the present work. Neto et al. [53] found that 19.9% of lines that are removed during a fix are related to refactoring operations and, therefore, their respective defect-introducing changes are likely false positives. Falessi and Moede [16] presented the Pilot Defects Prediction Dataset Maker (PDPDM), a desktop application to measure metrics for use in defect prediction. PDPDM avoids the use of outdated datasets, and it allows researchers and practitioners to create defect datasets without writing any code. Rodríguez-Pérez et al. [62] investigated the complex phenomenon of defect introduction and defect fix. They showed that less than 30% of defects can actually be traced to its origins by assuming that "a given defect was introduced by the lines of code that were modified to fix it." Our research complements the prior research in defect introduction by providing methods to retrieve the AVs. AVs can then be used to evaluate or improve approaches such as SZZ [10].

Extensive research has been invested in building and evaluating datasets for defect prediction. Shepperd et al. [67] investigated five studies that have used the NASA dataset for building defect prediction models. The goal of their work was to verify whether the different versions of the NASA dataset yield consistent results. Shepperd et al. [67] observed that different versions of the same dataset (e.g., NASA) may produce different results for defect prediction and, therefore, researchers should be cautious before selecting a dataset. Nam and Kim [52] proposed the Clustering and Labeling (CLA) and Clustering, Labeling, Metric Instance (CLAMI) approaches to automatically label unlabeled defect prediction datasets, relieving researchers from the manual effort. The approaches work based on the magnitude of metric values and obtain average prediction performances of around 0.64 (F-measure) and 0.72 (Area Under the receiver operating characteristic (ROC) Curve (AUC)).

Other studies focused on how to select repositories to mine. Nagappan et al. [50] combined ideas from representativeness and diversity, and introduced a measure called sample coverage, which is the percentage of projects in a population that are similar to the given sample. They concluded that studies should discuss the target population of the research (universe) and dimensions that potentially can influence the outcomes of the research (space). Gousios and Spinellis [26] proposed the Alitheia Core analysis platform, which pre-processes repository data into an intermediate format that allows researchers to provide custom analysis tools. Rozenberg et al. [63] proposed RepoGrams to support researchers in qualitatively comparing and contrasting software projects over time using a set of software metrics. RepoGrams uses an extensible, metrics-based, visualization model that can be adapted to a variety of analyses. Falessi et al. [19] presented Semi-auTomated

[4] http://svn.apache.org/viewvc?view=revision&revision=1100026.

REplicable projectS Selection (STRESS), a semi-automated and fully replicable approach that allows researchers to select projects by configuring the desired level of diversity, fit, and quality.

A significant effort has been invested in measuring the noise in defects repositories and its impact on the follow-up analyses. Bird et al. [5] found that bias is a critical problem that threatens both the effectiveness of processes that rely on biased datasets to build prediction models and the generalizability of hypotheses tested on biased data. Kim et al. [38] measured the impact of noise on defect prediction models and provided guidelines for acceptable noise levels. They also proposed a noise detection and elimination algorithm to address this problem. However, the noise studied and removed is supposed to be random. Herzig et al. [30] reported that 39% of files marked as defective actually never had a defect. They discussed the impact of this misclassification on earlier studies and recommended manual data validation for future studies. Rahman et al. [58] showed that size always matters just as much as bias direction, and, in fact, much more than bias direction when considering information-retrieval measures such as AUC, ROC and F-score. This indicates that, at least for prediction models, even when dealing with sampling bias, simply finding larger samples can sometimes be sufficient. Tantithamthavorn et al. [71] found that: (1) issue report mislabeling is not random; (2) precision is rarely impacted by mislabeled issue reports, suggesting that practitioners can rely on the accuracy of modules labeled as defective by models that are trained using noisy data; (3) however, models trained on noisy data typically achieve about 60% of the recall of models trained on clean data. Complementary to the aforementioned studies, we measure the extent of noise (i.e., classes mislabeled) and its impact on analyzing a repository in terms of features selection.

Another line of research in defect prediction has proposed the usage of a machine learning model to predict whether an upcoming change (i.e., commit) is defective or clean [23, 33, 36, 37, 79, 82]. This area of research was eventually coined as *Just-in-time* defect prediction (JIT). Kim et al. [37] proposed the usage of JIT models in their seminal work. In order to label their datasets, the authors used the output from SZZ. Fukushima et al. [23] and Kamei et al. [33] advanced the area and explored the usage of cross-project JIT models to help software projects without enough historical data to build their own models. In our work, we aim at retrieving AVs and verifying whether our methods can improve the accuracy on labeling defective classes. It is worth noting that we do not use our approaches for building JIT models as we envision to do so in future work.

Other researchers suggest that prediction models should be used, and validated according to their ability, to ranking modules with respect to their defect proneness [80, 81, 85, 86]. Yang et al. [80] proposed the use of Learning-to-Rank (LTR) algorithms to rank the defect proneness of software modules in their seminal work. Later, Yang et al. [81] expanded their seminal work to (i) apply the LTR method to datasets containing multiple releases; (ii) to perform more comparisons between the LTR method and other algorithms (e.g., algorithms based on least squares); and (iii) to investigate additional metrics to build and evaluate defect prediction models. Panichella et al. [54] proposed the usage of genetic alrogithms to tune the parameters of defect prediction to maximize the ratio between the number of defects and the required effort to inspect these defects. This ratio between defects found and required effort to inspect them can be coined as "cost." The authors observed that their approach significantly outperformed traditional models. Yu et al. [86] proposed a cost-sensitive support vector machine algorithm to improve ranking-oriented defect prediction models. For example, predicting the wrong rank for a module that contains a higher number of defects is worse than predicting the wrong rank for a module with less number of defects. Therefore, Yu et al.'s [86] approach is sensitive to such costs. The authors found that their approach outperformed approaches that are not cost-sensitive. Our work can potentially complement the rank-oriented defect prediction models in the sense that retrieving AVs can improve the

labeling of defective modules. Therefore, a potential future work is to build rank-oriented defect prediction models using datasets labeled by our proposed approach.

3 RESEARCH QUESTIONS

In this article, we investigate four research questions:

- **RQ1: Are AVs available and consistent?** Two recent SZZ studies [10, 83] recommend using AV to identify the origin of a defect and, hence, create defect datasets. However, how often do developers actually provide AVs in defect reports? In this research question, we investigate the extent to which AVs are usable, i.e., whether they are available and consistent.
- **RQ2: Do methods have different accuracy for labeling affected versions?** If AV of a defect is not available, then it must be retrieved. In this research question, we compare the accuracy of 10 different methods for labeling versions as affected or not by a defect. The 10 methods consist of 3 baseline methods belonging to the SZZ family and 7 new methods, which leverage the lifecycle information of defects.
- **RQ3: Do methods have different accuracy for labeling defective classes?** In order to mine a dataset of defects, it is important to have information about which class in which version is defective. Therefore, in this research question, we investigate which methods have the highest accuracy for labeling defective classes. We use commits' information to trace classes to defect-fixing changes and, hence, labeling specific classes as defective or not. We then merged this information with the information about affected versions (RQ2) to label classes in specific versions as defective or not. In other words, in this research question, we observe the accuracy of the realistic approach [83], in labeling classes, when the AVs are retrieved by different methods. This investigation is important as the accuracy of mining activities is highly correlated with the correctness of the datasets that are used [41, 66].
- **RQ4: Do methods lead to selecting different features?** To measure the level of impact on practitioners of the use of different methods, in this research question, we investigate the accuracy of methods in leading to accurate feature selection. Specifically, feature selection is the activity of identifying what features contribute the most in a prediction model for predicting whether a class is defective [7, 21, 27, 42, 64]. If a feature is selected as important, then it is strongly associated with the defectiveness of classes. As such, these important features can provide practitioners and researchers with knowledge on how to avoid future defects [2, 4, 15, 27, 39, 55, 57, 70, 76, 87]. However, inaccurate datasets may lead to the identification of the wrong features and, hence, to ineffective suggestions on how to avoid defects.

As the feature selection accuracy depends on the accuracy of labeling defective classes, which in turn depends on the accuracy of labeling affected versions, then RQ2 results impact RQ3 results, which, in turn, impacts RQ4 results. Since the actual extent of the differences across methods performances in specific research questions is currently unknown and could vary due to minor factors, then the existence of correlations across research questions does not decrease the value of each research question. For instance, the set of features identified (RQ4) could not vary, despite the variation of the methods' accuracy in defective class labeling (RQ3), due to the use of the specific set of features. For example, one feature could be selected regardless of the method used to develop the dataset, since the other features are uncorrelated to it anyhow.

Before reporting on the design and results of each of our research questions, we present the concepts that are shared across research questions.

We define a failure as an incorrect or unexpected result provided by the system or, more generally speaking, when the system behaves in an unintended way. Note that a failure is always caused by a defect, but a defect causes a failure only under specific conditions.

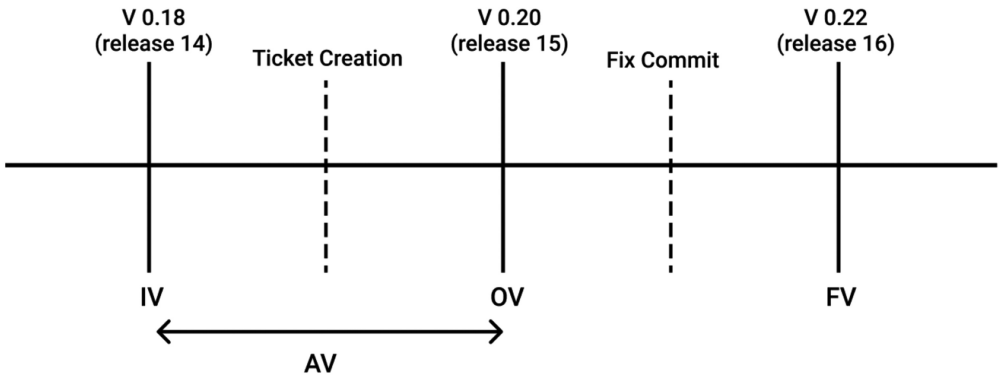


Fig. 2. Example of the life-cycle of a defect: Introduction Version (IV), Opening Version (OV), Fixed Version (FV), and Affected Versions (AV). Note, versions V0.19 and V0.21 were only "baselines" and not "user-intended" versions and, hence, were excluded.

Figure 2 illustrates the key terms while using the defect QPID-4462[5] as an example. The defect is first injected in the code at the Introducing Version (IV), i.e., the V0.18 version in Figure 2. Afterward, a failure is observed and a defect report is created to describe the defect. We refer to the version related to the creation of the defect report as the Opening Version (OV), i.e., the V0.20 version in Figure 2. Next, in a given future version, the defect is fixed by changes performed in one or more classes. We refer to the version related to the fix of the defect as the Fixing Version (FV), i.e., the V0.22 version in Figure 2. An AV is any version in which the defect could have caused a failure, i.e., any version *affected* by the defect. Thus, the AVs in our example are those in the range [IV, FV), i.e., the V0.18 and V0.20 versions in Figure 2. The V0.22 version is not an AV since it contains the fix and is not affected by the defect.

The OV is available in all defect reports as it is generated by the issue tracker at the creation of the report. The FV is available in defect reports where developers have mentioned the defect report ID in the log of the commit that fixes the defect. For example, commit *732ab160852f943cd847646861dd48370dd23ff3* is the last commit including *[QPID-4462]* in its log. Since this commit was performed at *2013-03-31T21:51:49+00:00*, we can infer that it has been performed between versions V0.20 and V0.22.

Our intuition, that defects have a stable life cycle in terms of the proportion of the number of versions required to discover and to fix a defect, actually means that FV-OV is proportional to FV-IV.

3.1 RQ1: Are AVs available and consistent?

3.1.1 Design. In this section, we report the design and result of our first research question.

Dependent variables. Our dependent variable is the percentage of *available & consistent* AVs. An AV is *available* if it is provided in the report related to a defect (i.e., the defect report). An AV information is *consistent* when the earliest AV occurs before the OV. The rationale is that the defect must have affected a version that occurred at least at the moment when the defect report had been created. That is, a defect cannot have been injected *after* the related failure had been observed.

Measurement Procedure. To measure the availability and consistency of AVs, we follow the following nine steps:

[5] https://issues.apache.org/jira/browse/QPID-4462.

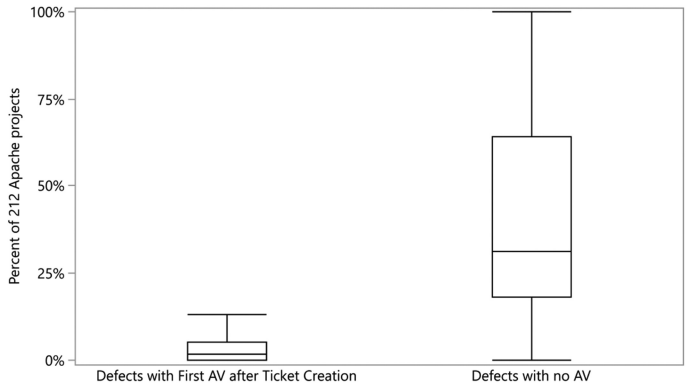


Fig. 3. Distribution of 212 Apache projects having a specific proportion of defect reports with an unreliable AV (left side) or without the AV (right side).

(1) We retrieve the Jira and Git URL of all existing Apache projects.[6] We focused on Apache projects rather than GitHub projects because Apache projects have a higher quality of defect annotation and are unlikely to be toy projects [49]. Finally, Apache projects use Jira as their issue tracker, which allows us to study the consistency and availability of AV information.
(2) We filter out projects that are not tracked in Jira or not versioned in Git. This leads to 212 projects.
(3) As recently done by Borg et al. [45], for each project, we count the number of issue reports by performing the following query to the Jira repository: *Type == "defect" AND (status == "Closed" OR status == "Resolved") AND Resolution == "Fixed"*. This gave us a total of about 235,000 defects.
(4) We exclude issue reports not having a related Git commit fixing it.
(5) We exclude defects that are not post-release. Post-release defects are also known in the industry as production defects, i.e., defects that caused failures observed by users. Thus, a defect that is injected and fixed in the same version is not a post-release defect. For brevity, in the remainder of this article, we refer to post-release defects simply as defects. After steps 4 and 5, we are left with a total of 125,000 defects.
(6) For each defect report, we check its AV **availability**, i.e., the presence of the AV field, by performing the following query to the Jira repository: Affect Version ≠ "Null". Thus, each issue report is tagged as available or unavailable.
(7) For each defect report, we check its AV **consistency**, i.e., if IV ≤ OV.
(8) For each project, we compute the percentage of unreliable and unavailable AV.
(9) Finally, we observe the distribution of 212 Apache projects in terms of percentages of defects having available, and available and consistent AV.

3.1.2 RQ1: Results. Figure 3 reports the distribution of 212 Apache projects having a specific proportion of defects with an unreliable AV (left side) or without the AV (right side). According to Figure 3, most of the projects have more than 25% of defect reports without any AV. We also measured the total number of closed defect reports linked with git commits in the 212 Apache projects, which resulted to be 125,860. Of these, 63,539 defect reports (51%) resulted in not having

[6] https://people.apache.org/phonebook.html.

or having inconsistent AVs. Thus, we can claim that *in most of defect reports, we cannot use the AVs and, hence, we often need an automated method for retrieving AVs.*

3.2 RQ2: Do Methods have Different Accuracy for Labeling Affected Versions?

3.2.1 Design. In this section, we report the design and result of our second research question. Since AVs are in the range [IV, FV), and since we always know FV, retrieving the AVs of a defect actually translates into estimating its IV. One approach to estimate the IV of a defect is to employ the SZZ algorithm. The oldest defect-introducing commit produced by SZZ can be considered as the IV, whereas all other defect-introducing commits can be used to label the consecutive versions before the defect-fixing commit as other AVs (the defect-fixing commit itself is not considered in the labeling process, of course). However, existing researches have highlighted substantial limitations of the SZZ approach [10, 60, 62].

We investigate the following null hypothesis in this RQ:

— H_{10}*: different methods obtain the same accuracy for labeling AVs.*

Independent variables. Our independent variable is the method used to retrieve the AV, i.e., to label a version as affected or not by a specific defect. In this work, we present three variants of a new approach called *Proportion* to label affected versions. The Proportion approach assumes a stable proportion (P), among defects of the same project, between the number of affected versions between IV and FV, and the number of versions between OV and FV. The rationale is that the life-cycle might be consistent among defects of the same projects. Thus, in some projects, defects require a number of versions to be found and another number to be fixed. Our intuition is that the proportion among these numbers is somehow stable across defects of the same project. Of course, defects of the same projects may vary and, hence, we do not expect this method to be perfectly accurate. Since FV and OV are known for every defect, the idea is to compute P on previous defects, and then use it for defect reports where AVs are not available nor consistent. Thus, we define P as $(FV - IV)/(FV - OV)$. Therefore, we can calculate the IV as $FV - (FV - OV) * P$. Among the possible ways to use Proportion, we propose the following three methods:

— **Proportion_Incremental:** It consists of computing P of the current defect as the average P on past defects of the same project. This approach has the advantage of using the highest amount of information, within the same project of the current defect, available at a given point in time. The assumptions of this approach are that (1) P does not change over time within the same project and that (2) there is enough information on previous defects of the project, i.e., there are enough previous defects to represent the true project average P. Specifically, in this method, we ordered the defects by fix date. For each version R within a project, we used the average P among defects fixed in versions 1 to R-1. Using the example in Figure 2, the *P_Increment*, computed as the average P among defects in versions 1 to 15, is 1.7775. Therefore, $IV = 16 - (16 - 15) * 1.7775$, which is 14.2225. Hence, this method would correctly identify 0.20 as affected (true positive), but incorrectly classify 0.18 as not affected (false negative).

— **Proportion_ColdStart:** It consists of computing P of the current defect as the average P on defects of other projects. This approach has the advantage of using the knowledge related to other projects and it is particularly useful for the first defects in a project, i.e., when there are not enough previous defects to represent the true project average P. The assumption of this approach is that P is stable across defects of different projects. Specifically, for each studied project, we compute the average P across all defects within the project. We label each of these projects as $P_PROJECT$ where $PROJECT$ is the project's ID. Next, for each

project, we take the median of the *P_PROJECT* values among all other projects to use as the *P_ColdStart*. Using the example in Figure 2, the indexes of the 0.18, 0.20, and 0.22 versions are 14, 15, and 16, respectively. The *P_ColdStart* computes as the median of the other *P_PROJECT* proportions, as 1.8089. Therefore, $IV = 16 - (16 - 15) * 1.8089$, which is 14.1911. Hence, this method would correctly identify 0.20 as affected (true positive), but incorrectly classify 0.18 as not affected (false negative).

— **Proportion_MovingWindow:** It consists of computing P of the current defect as the average P on a limited number of recent defects of the same project. This approach has the advantage of relaxing the assumption that P does not change over time within the same project as we limit the amount of time where this assumption needs to hold, as opposed to Proportion_Incremental where the assumption needs to hold for the entire length of the project. The assumptions of this approach are that (1) the length of the window is big enough to contain sufficient previous defects to represent the true project average P, and (2) the window is small enough to exclude defects that are different to the current one. Therefore, the hard part in implementing the Proportion_MovingWindow method is in defining the right length of the window. Specifically, we ordered the defects by their fix date. For each defect within a project, we used the average P among the last 1% of fixed defects. We chose a 1% moving window length as a tradeoff between the ability to react to changes in the data and the resilience to outliers. Using the example in Figure 2, the *P_MovingWindow* is computed as the average P among the last 1% of defects. There are 1,192 defects in the project of Figure 2. Therefore, there are around 12 defects at the 1% of defects. The average P among the last 12 fixed defects is 2.167. Therefore, $IV = 16 - (16 - 15) * 2.167$, which is 13.833. Hence, this method would correctly identify 0.18 and 0.20 as affected (true positive), giving 100% accuracy for this defect.

Moreover, a simplistic way to apply Proportion is to assume IV equals OV. Therefore, we propose the following simplistic method:

— **Simple**: It simply assumes that the IV corresponds to OV. The rationale is that, by definition, all versions from OV to FV (not including FV) are AV. However, versions before OV can also be AV. Therefore, we expect this heuristic to achieve a 100% Precision but a lower Recall. Specifically, this heuristic would identify 0.20 as IV in Figure 2. Therefore, it would miss 0.18 (false negative) and would correctly identify 0.20 (true positives) as AVs.

Furthermore, we considered as baseline the well-known SZZ approach. As previously discussed in Section 2, SZZ is an algorithm that, given a fix commit, determines the possible defect-introducing commits. In our methods, we assume the oldest defect-introducing commit to be the IV. Specifically, among the possible ways to use SZZ, we considered the following methods:

— **SZZ_Basic**: We use the SZZ algorithm [69] to determine when the defect has been introduced, and we assume as AVs all versions between the IV and the FV (not including FV). In the example in Figure 2, SZZ_B identified three defect-introducing commits with the following dates: 2012-05-19T08:54:25, 2012-10-06T05:38:51, and 2012-11-05T10:03:36. Among these, 2012-05-19T08:54:25 is the oldest date, which falls into version 0.18 labeled as the IV. Therefore, the AVs are 0.18 and 0.20. Versions 0.18 and 0.20 were correctly identified as affected (true positives), and, therefore, this method receives 100% accuracy for this defect.

— **SZZ_U**: We rely on an open implementation of SZZ by Borg et al. [6] and we set the depth to one. This SZZ implementation does not discard cosmetic changes (since it supports all programming languages). However SZZ_U uses Jaccard distances to map moving lines. In the example in Figure 2, SZZ_U identified one defect-introducing commit dated 2012-05-

18T20:54:25, which falls into version 0.16 labeled as the IV. Therefore, the AVs are 0.16, 0.18, and 0.20. Versions 0.18 and 0.20 were correctly identified as affected (true positives) and version 0.16 was incorrectly identified as affected (false positives).

— **SZZ_RA:** We use a refactoring-aware SZZ algorithm implemented by Da Costa [10]. This algorithm tracks defect-introducing commits and filters out refactoring operations. However, this implementation only analyzes java files, so the defect-introducing commits for non-java files are determined by SZZ_U. In the example in Figure 2, SZZ_RA identified one defect-introducing commit dated 2012-05-18T16:54:25, which falls into version 0.16 labeled as the IV. Therefore, the AVs are 0.16, 0.18, and 0.20. Versions 0.18 and 0.20 were correctly identified as affected (true positives) and version 0.16 was incorrectly identified as affected (false positives).

Finally, instead of using the SZZ_X methods as is, we improved them by using the information provided by the Simple method. Specifically, we created methods **SZZ_X+** by merging each *SZZ_X* with the *Simple* method: a version is affected if SZZ_X labeled it as affected or Simple labeled it as affected. Hence, we are merging the defects' life cycle information with the SZZ-based method. The rationale is that if Simple labels a version as affected, then the version is actually affected by definition. To illustrate how this works, we will use a new example, WICKET-4071.[7] The AVs indicated on this defect report are: 1.4.6, 1.4.7, 1.4.8, 1.4.19, 1.4.10, and 1.5-M1. The OV is 1.4.8 and FV is 1.5-M1. Simple would classify versions 1.4.8, 1.4.19, 1.4.10, and 1.5-M1 as affected (true positives) and would miss versions 1.4.6 and 1.4.7 (false negatives). SZZ_B would classify 1.4.10 and 1.5-M1 as affected (true positives) and miss versions 1.4.6, 1.4.7, 1.4.8, and 1.4.19 (false negatives). However, SZZ_B+ would classify versions 1.4.8, 1.4.19, 1.4.10, and 1.5-M1 as affected (true positives) and would miss versions 1.4.6 and 1.4.7 (false negatives).

In conclusion, in this article, we analyze the accuracy of three already known methods (SZZ_X) and seven new methods (Proportion_X, SZZ_X+, and Simple)

Dependent variables. Our dependent variable is the accuracy for labeling versions of a project as affected, or not, by a defect. We use the following set of metrics:

— True Positive (TP): The version is actually affected and is labeled as affected.
— False Negative (FN): The version is actually affected and is labeled as non-affected.
— True Negative (TN): The version is actually non-affected and is labeled as non-affected.
— False Positive (FP): The version is actually non-affected and is labeled as affected.
— **Precision**: $\frac{TP}{TP+FP}$
— **Recall**: $\frac{TP}{TP+FN}$
— **F1**: $\frac{2*Precision*Recall}{Precision+Recall}$
— Cohen's **Kappa**: A statistic that assesses the classifier's performance against random guessing [9]. $Kappa = \frac{Observed-Expected}{1-Expected}$ where
  — Observed: The proportionate agreement. $\frac{TP+TN}{TP+TN+FP+FN}$
  — Expected: The probability of random agreement. $P_{Yes} + P_{No}$ where
    * P_{Yes}: Probability of positive agreement. $\frac{TP+FP}{TP+TN+FP+FN} * \frac{TP+FN}{TP+TN+FP+FN}$
    * P_{No}: Probability of negative agreement. $\frac{TN+FP}{TP+TN+FP+FN} * \frac{TN+FN}{TP+TN+FP+FN}$
— **Matthews** Correlation Coefficient: $\frac{TP*TN-FP*FN}{\sqrt{(TP+FP)(TP+FN)(TN+FP)(TN+FN)}}$

[7] https://issues.apache.org/jira/browse/WICKET-4071.

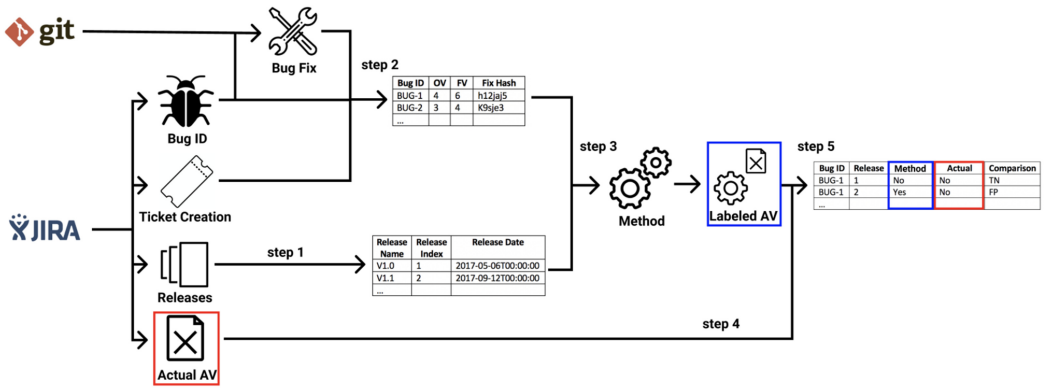


Fig. 4. The process to measure the accuracy of methods in labeling affected versions.

Since we have binary classifications that are thresholds independent, we do not use Area Under the Receiver Operating Characteristic metric.

Measurement procedure. We began by selecting the projects with the highest proportion of usable (i.e., available and consistent) AVs. We selected projects with at least 100 defects that were linked with git and contained available and consistent AVs. Then, we filtered out projects with less than six versions. Lastly, we filtered out projects where the percent of available and consistent AVs are less than 50%. This left us with 76 projects. For each project, we followed the steps below. See Figure 4 for an overview of this process.

(1) We retrieved the versions of the project and their release dates from Jira. We numbered these versions beginning with the oldest version as version 1.
(2) We used the defects, of which their reports provided available and consistent AVs in RQ1. For each defect, we determined the IV (i.e., the version of the first AV labeled by Jira), OV (i.e., the version of the ticket creation), FV (i.e., the fix version), and the fix commit hash by Git. We ordered the defects by fix date.
(3) For each defect, we labeled versions 1 to FV as affected or not by each of the following methods:
   (a) **Simple:**
      (i) We set IV equal to OV.
      (ii) For each defect, we label each version before the IV as not affected. We label each version from the IV to the FV as affected. The FV is labeled not affected.
   (b) **SZZ:**
      (i) We ran each SZZ implementation on the project by supplying the Git directory and a list of defects and their fix commit.
      (ii) For each defect, SZZ outputs all possible defect-introducing commits. We compute the corresponding version for each defect-introducing commit. We chose the oldest version to be the IV.
      (iii) For each defect, we label each version before the IV as not affected. We label each version from the IV to the FV as affected. The FV is labeled not affected.
   (c) **Proportion_ColdStart:**
      (i) We computed the average P across the project's defects, i.e., $P = (FV - AV)/(FV - OV)$. If FV equals OV, then $FV - OV$ is set to one to avoid divide by zero cases.
      (ii) We computed the $P_ColdStart$, i.e., the median P of all other projects.

(iii) For each defect, we computed the IV as $IV = (FV - OV) * P_ColdStart$. If FV equals OV, then IV equals FV. However, recall we excluded defects that were not post-release. Therefore, we set $FV - OV$ equal to 1 to assure IV is not equal to FV.
(iv) For each defect, we label each version before the IV as not affected. We label each version from the IV to the FV as affected. The FV is labeled not affected.

(d) **Proportion_Increment:**
(i) For each version R, we computed $P_Increment$ as the average P among defects fixed in versions 1 to R-1.
(ii) We used the P_ColdStart for P_Increment values containing less than five defects on average.
(iii) For each defect in each version, we computed the IV as $IV = (FV - OV) * P_Increment$. If FV equals OV, then IV equals FV. However, recall we excluded defects that were not post-release. Therefore, we set $FV - OV$ equal to 1 to assure IV is not equal to FV.
(iv) For each defect, we label each version before the IV as not affected. We label each version from the IV to the FV as affected. The FV is labeled not affected.

(e) **Proportion_MovingWindow:**
(i) For each defect, we computed P_MovingWindow as the average P among the last 1% of defects. The defects are ordered by their fix date.
(ii) We used the P_ColdStart for P_MovingWindow values containing less than 1% of defects on average.
(iii) For each defect, we computed the IV as $IV = (FV - OV) * P_MovingWindow$. If FV equals OV, then IV equals FV. However, we excluded defects that were not post-release. Therefore, we set $FV - OV$ equal to 1 to assure IV is not equal to FV.
(iv) For each defect, we label each version before the IV as not affected. We label each version from the IV to the FV as affected. The FV is labeled not affected.

(f) **+:**
(i) For each SZZ method, we combined it with Simple. For each defect, we labeled each version as affected if SZZ_X or Simple labeled the version as affected.

(4) We determined the observed/actual AV by looking at Jira values. We label each version before the IV, as labeled by Jira developers, as not affected. We label each version from the IV to the FV, as labeled by Jira developers, as affected. The FV, as labeled by Jira developers, is labeled not affected.
(5) For each method, we compared the classification to the actual classification and computed the TP, TN, FP, FN, Precision, Recall, F1, Matthews, and Kappa across the project's version-defect pairs.

Hypothesis testing. To test hypothesis H_{10}, we used the Kruskal–Wallis test [43], which is a non-parametric test (i.e., a normally distributed data is not required) to check whether three or more distributions are significantly different. This test is necessary because we compare 10 distributions of values (i.e., one for each studied method). For example, we compare whether our 10 studied methods for estimating AVs significantly differ in terms of precision or recall. We chose the non-parametric Kruskal–Wallis because our performance metrics (e.g., precision or recall) do not follow a normal distribution (as we noted when performing Shapiro–Wilk tests [65]). Therefore, our approach is compliant to the suggestion to avoid using ScottKnottESD in case of not normal distributions [29].

Although Kruskal–Wallis provides an indication of whether a significant difference between distributions exists, it does not indicate which specific pairs of distributions are significantly

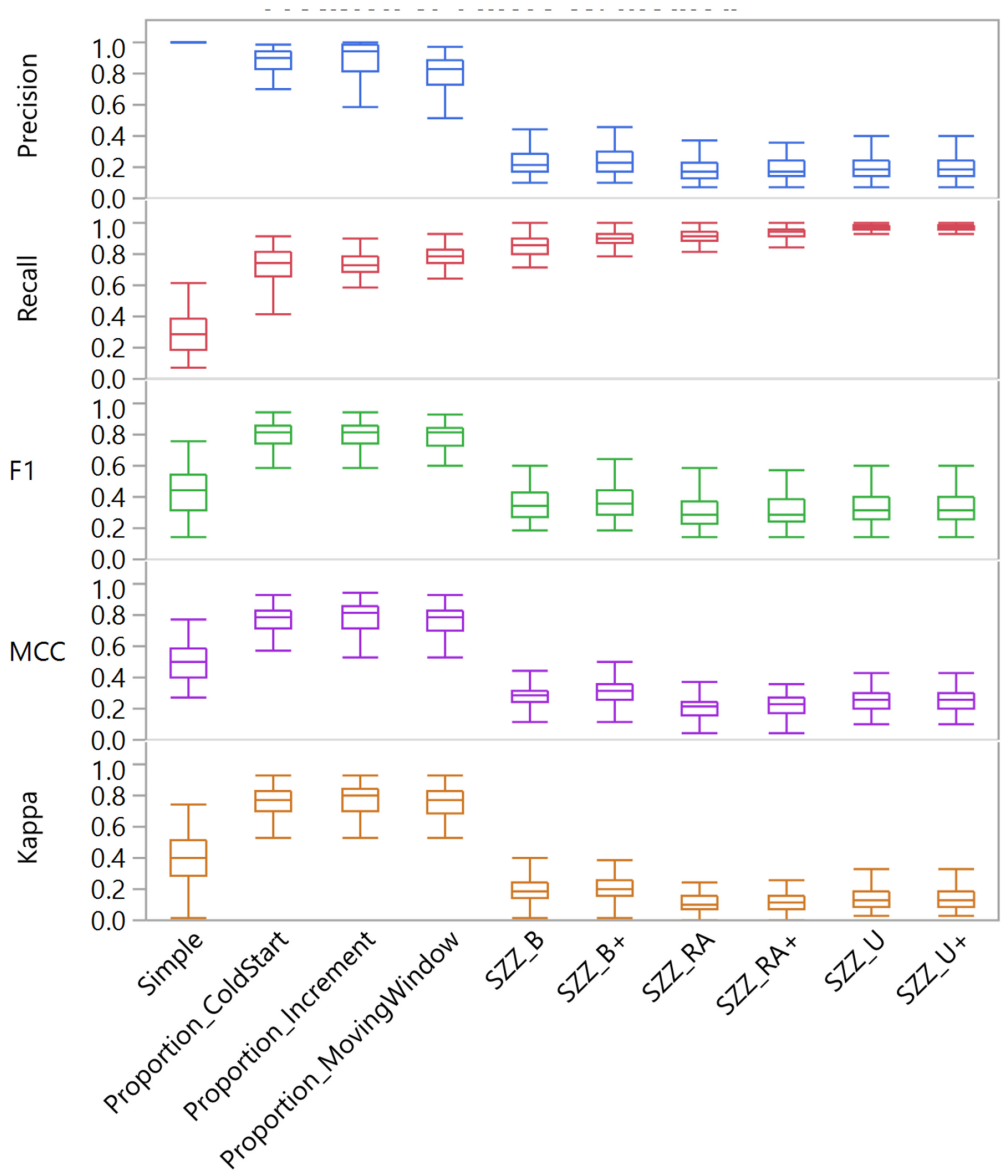


Fig. 5. Distribution, across 76 Apache projects, of Precision, Recall, F1, MCC, and Kappa, of different methods in labeling AV.

different between each other. For this purpose, we perform a follow-up Dunn test [13], which is a *post-hoc* test to indicate which specific pairs of distributions significantly differ between each other. To account for the chance of errors due to multiple comparisons, we perform a Holm-Bonferroni correction of our $p - values$ [31].

3.2.2 RQ2: Results. Figure 5 reports the distribution, across 76 Apache projects, of Precision, Recall, F1, Matthews correlation coefficient (MCC), and Kappa, of different methods in labeling AV. According to Figure 5:

Table 1. Dunn's Results for the *Precision* Values of Hypothesis H_{10}

Method	Mean Precision	Dunn's test Rank
Simple	1.0	1
Proportion Increment	0.90	2
Proportion Cold Start	0.89	2
Proportion Moving Window	0.81	3
SZZ_B+	0.26	4
SZZ_B	0.25	4
SZZ_U+	0.22	5
SZZ_U	0.22	5
SZZ_RA+	0.21	5
SZZ_RA	0.20	5

Table 2. Dunn's Test Results for the *Recall* Values of Hypothesis H_{10}

Method	Avg. Recall	Dunn's Test Rank	Comment
SZZ_U+	0.97	1	
SZZ_U	0.97	1	
SZZ_RA+	0.94	1.5	Significantly lower than SZZ_U+
SZZ_RA	0.91	2	
SZZ_B+	0.89	2	
SZZ_B	0.85	2.5	Significantly lower than SZZ_RA
Proportion Moving Window	0.78	3	
Proportion Increment	0.73	3	
Proportion Cold Start	0.72	3	
Simple	0.30	4	

We increased the rank by a half whenever a method is significantly different from only one other method within the group.

- —All the Proportion methods have a higher Precision and composite accuracy (F1, MCC, and Kappa) than all SZZ methods.
- —Simple has a higher Precision and composite accuracy (F1, MCC, and Kappa) than all SZZ methods.
- —SZZ_U has the highest Recall than all other methods.
- —SZZ_B+ has the highest Precision and the highest composite accuracy (F1, MCC, and Kappa) than any other SZZ method.
- —The method with the highest precision is Simple. This is true by definition.
- —There is no single dominant method among the Proportion methods. For instance, Proportion_Increment provides the highest Precision, F1, and Kappa, and it dominates Proportion_ColdStart. Proportion_MovingWindow provides the highest Recall (among Proportion methods) and MCC.

Our statistical results on the 76 Apache projects reveal that the differences between our studied methods in terms of the accuracy metrics are statistically significant for H_{10} (i.e., our Kruskall–Wallis and Dunn's tests yielded $p - values < 0.05$). Therefore, our results reveal that the proportional methods have significantly better accuracy values compared to the studied SZZ-based methods. Tables 1–3 show the results of the Dunn's tests for the precision, recall, and F1 metrics,

Table 3. Dunn's Test Results for the *f1* Values of Hypothesis H_{10}

Method	Avg. F1	Dunn's Test Rank	Comments
Proportion Increment	0.80	1	
Proportion Moving Window	0.80	1	
Proportion Cold Start	0.79	1	
Simple	0.44	2	
SZZ_B+	0.39	2	
SZZ_B	0.37	2	
SZZ_U+	0.35	2	
SZZ_U	0.35	2	
SZZ_RA+	0.32	2	
SZZ_RA	0.32	2.5	Significantly lower than Simple

We increased the rank by a half whenever a method is significantly different from only one other method within the group.

respectively. For each table, we show (i) the methods, (ii) the *mean* accuracy, and (iii) the *rank* of the method. The *rank* is based on whether the Dunn's test provided a significant $p - value$ for a given comparison. For example, in Table 1, the *Simple* approach is placed at the top rank because (a) it has the highest mean value and (b) the Dunn's tests yielded significant $p - values$ for every pair-wise comparison between the *Simple* method and the other methods. On the other hand, the *Proportion Increment* and *Proportion Cold Start* methods are placed at the second rank because the Dunn's test revealed that these two specific methods do not have statistically different precisions. However, both the *Proportion Increment* and *Proportion Cold Start* methods have significant $p - values$ when compared to the other methods that fell in lower ranks.

According to our observations, the *Proportion Increment*, *Proportion Moving Window*, and *Proportion Cold Start* methods are all in the 1st rank in terms of F1. However, the *Proportion Increment* and *Proportion Cold Start* methods yield significantly better precision values (i.e., $p - values < 0.05$) compared to the *Proportion Moving Window* method (see Table 1). Interestingly, even the *Simple* method (which fell in the 2nd rank; see Table 3) significantly outperforms the SZZ_RA method.

3.3 RQ3: Do Methods have Different Accuracy for Labeling Defective Classes?

3.3.1 Design. We propose the following hypothesis for this RQ:

— H_{20}: *different methods obtain the same accuracy for labeling classes.*
Our empirical procedure was inspired by Ref. [11], and it is detailed in the following sections.

Independent variables. The independent variable is represented by the same methods shown in RQ2. However, in this research question, the retrieved AVs (as performed in RQ2) are used to label classes as defective or not.

Dependent variables. The dependent variables are the same accuracy metrics presented in RQ2, with the only difference being that the unit upon which the accuracy is computed is the defectiveness of a class in a version. If at least one defect impacts the version-class pair, then the version-class pair is labeled as defective. This is demonstrated in Figure 6 where F1.java is deemed defective because it was touched by the fix for defect-3 in version 1 (i.e., at least one defect-fix touched F1.java in version 1).

In order to better explain the difference between RQ3 and RQ2, let's consider the case of methods A, B, and C, and a class that was affected by three defects in a certain version. Suppose that A is able to identify that the class was affected by one defect, B, by three defects, and C, by four

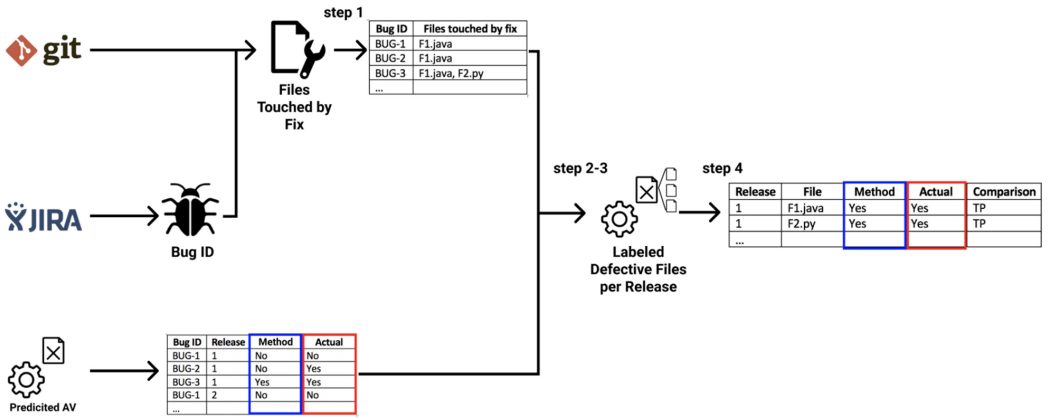


Fig. 6. The process to measure the accuracy of methods in labeling defective classes.

defects. In this example, all three methods correctly identify the class in the version as defective and, therefore, all three methods result with perfect accuracy. However, for the purpose of RQ2, method B has a higher accuracy than methods A and C. The following metrics have been redefined for this RQ:

- —True Positive (TP): The class in a version is actually defective and is labeled as defective.
- —False Negative (FN): The class in a version is actually defective and is labeled as non-defective.
- —True Negative (TN): The class in a version is actually non-defective and is labeled as non-defective.
- —False Positive (FP): The class in a version is actually non-defective and is labeled as defective.

Measurement procedure. Figure 6 describes the process we use to label a class in a version as defective or not. The process is identical to what Yatish et al. [83] coined as the realistic approach. The only difference is that the AV is assumed to be unavailable and, hence, it is retrieved by using a certain proposed method (see RQ2). The process consists of three steps:

(1) For each defect in RQ2, we computed a list of classes touched by the fix commit.
(2) For each method in RQ2, we labeled each version-class pair as defective if the version of the pair was determined to be an AV of at least one defect in RQ2 and the defect-fix commit of that defect involved the analyzed class. Otherwise, the version-class pair was labeled as not defective.
(3) We determined the observed/actual defectiveness of each version-class pair. To this end, we labeled each version-class pair as defective if the version of the pair was indicated as an AV of at least one defect by Jira developers themselves, and the defect-fix commit of that defect touched the class. Otherwise, the version-class pair was labeled as not defective. To identify which commit is related to which defect, we looked for the ticket's ID reported in the comment of the commit. For instance, consider the case of the defect ticket "QPID-4462" . All classes touched by all commits reporting the string "QPID-4462" are considered affected by defect "QPID-4462" .
(4) For each proposed method, we compared its classifications to the observed/actual classification. Next, we computed the TP, FN, TN, FP, Precision, Recall, F1, Matthews, and Kappa metrics across the projects.

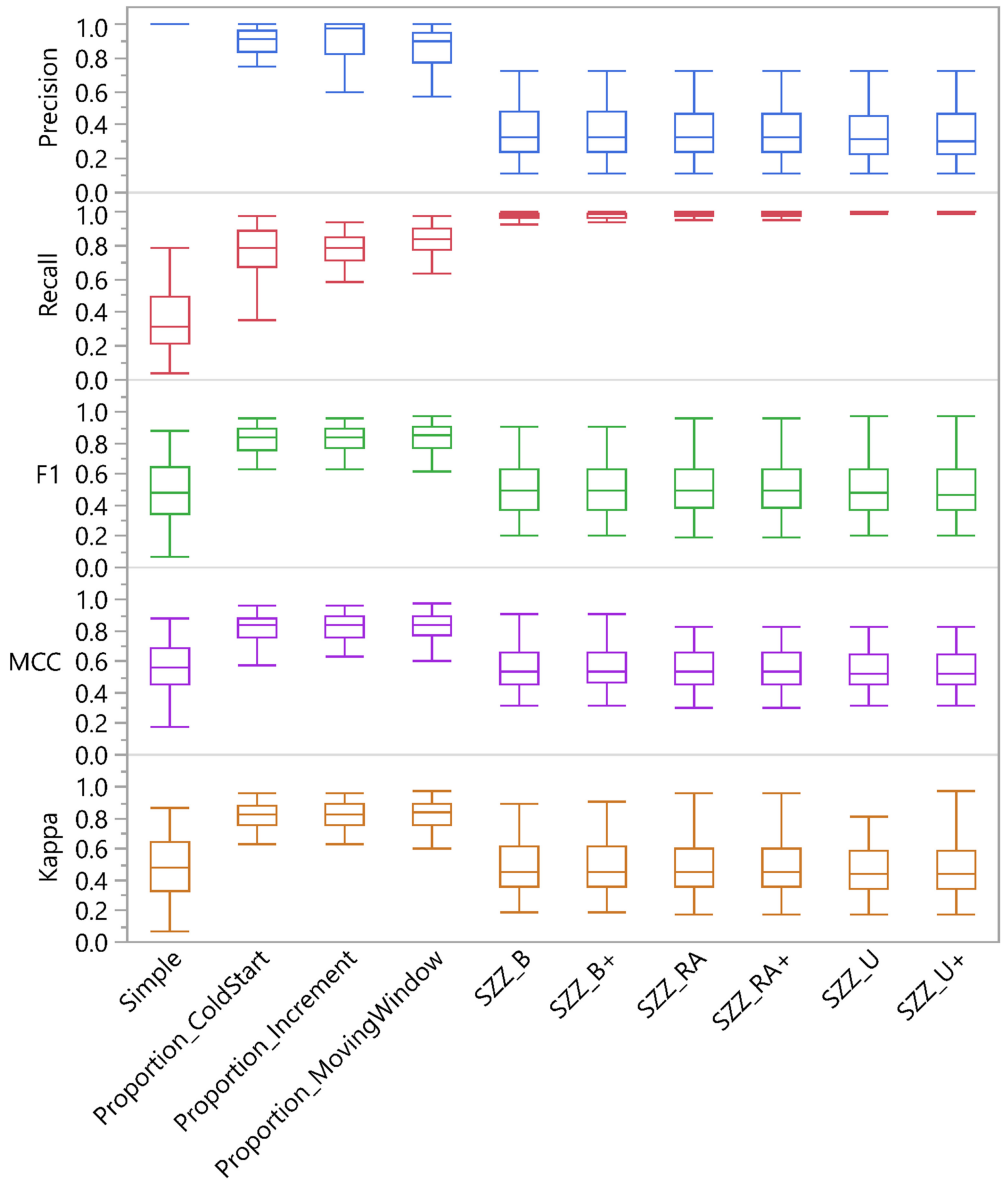


Fig. 7. Distribution, across 76 Apache projects, of Precision, Recall, F1, MCC, and Kappa, of different methods in labeling defective classes.

Hypothesis testing. To test hypothesis H_{20}, we use the same statistical machinery used in RQ2. We perform Kruskal-Wallis tests to check whether the distributions are significantly different. Next, we perform Dunn tests to identify the specific pairs of distributions that are significantly different between themselves.

3.3.2 RQ3: Results. Figure 7 reports the distribution, across 76 Apache projects, of Precision, Recall, F1, MCC, and Kappa, of different methods for labeling defective classes. According to Figure 7:

Table 4. Dunn's Test Results for the *Precision* Values of Hypothesis H_{20}

Method	Mean Precision	Dunn's Test	Comments
Simple	1.0	1	
Proportion Increment	0.91	1	
Proportion Cold Start	0.90	1.5	Significantly lower than Simple
Proportion Moving Window	0.86	1.5	Significantly lower than Simple
SZZ_B+	0.38	2	
SZZ_B	0.37	2	
SZZ_RA+	0.37	2	
SZZ_RA	0.37	2	
SZZ_U+	0.36	2	
SZZ_U	0.36	2	

We increased the rank by a half whenever a method is significantly different from only one other method within the group.

Table 5. Dunn's Test Results for the *Recall* Values of Hypothesis H_{20}

Method	Mean Recall	Dunn's Test Rank
SZZ_U	0.99	1
SZZ_U+	0.99	1
SZZ_B+	0.97	2
SZZ_RA	0.97	2
SZZ_RA+	0.97	2
SZZ_B	0.96	2
Proportion Moving Window	0.81	3
Proportion Increment	0.76	3
Proportion Cold Start	0.76	3
Simple	0.36	4

— All the proportional methods have a higher Precision and composite accuracy (F1, MCC, and Kappa) compared to all SZZ methods. Therefore, we can claim that *labeling classes using defects' life cycle information is in overall and, in average, more accurate than the studied SZZ methods.*
— SZZ_U has the highest Recall than all other methods.
— SZZ_B+ has a higher Precision and lower Recall than any other SZZ method.
— SZZ_B+ has a higher composite accuracy (F1, MCC, and Kappa) than Simple and any other SZZ method.
— *The Proportion_MovingWindow method dominates all methods on all composite accuracy (F1, MCC, and Kappa).*

Our statistical results (i.e., Kruskal–Wallis and Dunn's tests) reveal that hypothesis H_{20} can be rejected. Therefore, our proportion-based methods for labeling defective classes significantly outperform the studied SZZ-based methods in terms of the studied accuracy metrics (i.e., precision, recall, F1, kappa, and MCC). Tables 4–6 show our results yielded by the Dunn's tests. Indeed, we observe that all the proportion-based methods fall in the 1st rank of $F1$, whereas the SZZ-based methods and the *Simple* method fall in the 2nd rank of $F1$, obtaining a statistically significant lower performance.

Table 6. Dunn's Test Results for the $F1$ Values of Hypothesis H_{20}

Method	Mean F1	Dunn's Test Rank
Proportion Moving Window	0.83	1
Proportion Increment	0.82	1
Proportion Cold Start	0.81	1
SZZ_B+	0.52	2
SZZ_B	0.51	2
SZZ_RA	0.51	2
SZZ_RA+	0.51	2
SZZ_U+	0.51	2
SZZ_U	0.51	2
Simple	0.50	2

When comparing the proportion-based methods, we note that the *Proportion Increment* method yields precision values that are statistically similar to the *Simple* method, which fell in the first rank. On the other hand, the *Proportion Cold Start* and *Proportion Moving Window* methods yield precision values that are statistically lower compared to the *Simple* method. Therefore, the *Proportion Increment* method likely produces better precision values than the *Proportion Cold Start* and *Proportion Moving Window* methods.

3.4 RQ4: Do Methods Lead to Selecting Different Features?

3.4.1 Design. We propose the following hypothesis:

—H_{30}: *different methods lead to the same level of accuracy for feature selection.*

Independent variables. The independent variable is represented by the same methods used in RQ2 and RQ3. In RQ4, we use the labeled classes in RQ3 to select features.

Dependent variables. Our dependent variable is the accuracy for selecting features [27, 42]. We compare which features are selected on the same dataset when created by our different studied methods. The following metrics have been redefined for this RQ:

—True Positive (TP): The feature is selected in the actual repository, and it is selected in the repository generated by a method.
—False Negative (FN): The feature is selected in the actual repository, and it is not selected in the repository generated by a method.
—True Negative (TN): The feature is not selected in the actual repository, and it is not selected in the repository generated by a method..
—False Positive (FP): The feature is not selected in the actual repository, and it is selected in the repository generated by a method.

As features, to be selected, we used 17 well-defined product and project features that have been shown to be useful for defect prediction [12, 17]. Table 7 details the set of features.

Measurement Procedure. For each project, we compute the features in Table 7 as shown in Figure 8 and detailed in four steps.

(1) For each project, we begin by removing the last 50% of versions due to the fact that classes snore as described by Ahluwalia et al. [1].
(2) For each project P, we compute the features as described in Table 7 for each version-class pair.

Table 7. Defect Prediction Features

Metric	Description
Size	Lines of code (LOC)
LOC Touched	Sum over revisions of LOC added + deleted
NR	Number of revisions
Nfix	Number of bug fixes
Nauth	Number of authors
LOC Added	Sum over revisions of LOC added
MAX LOC Added	Maximum over revisions of LOC added
AVG LOC Added	Average LOC added per revision
Churn	Sum over revisions of added—deleted LOC
Max Churn	Maximum churn over revisions
Average Churn	Average churn over revisions
Change Set Size	Number of files committed together
Max Change Set	Maximum change set size over revisions
Average Change Set	Average change set size over revisions
Age	Age of Release
Weighted Age	Age of Release weighted by LOC touched

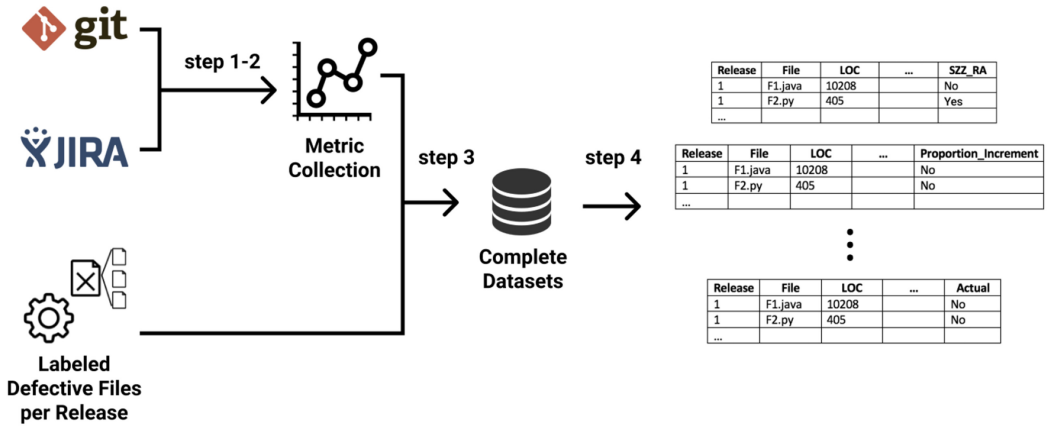


Fig. 8. The process of creating the Complete datasets for each project and method.

(3) For each of the methods M, we combined their produced AV datasets with the version-class pair's defectiveness (as computed in RQ3), which we labeled as P_M_Complete.
(4) For each version R within a project, we created a dataset including all version-class pairs with versions 1 to R labeled P_M_R_Complete. This dataset uses the defectiveness computed by method M in RQ3.

Afterward, we analyze which features are selected. Figure 9 reports an overview of the approach used in this RQ to measure the accuracy of methods for accurate feature selection. This approach consists of three steps:

(1) For each dataset P, for each version R, we set the class defectiveness according to each method M, and we perform on P_M_R an Exhaustive Search Feature Selection[8] using

[8]https://weka.sourceforge.io/doc.packages/attributeSelectionSearchMethods/.

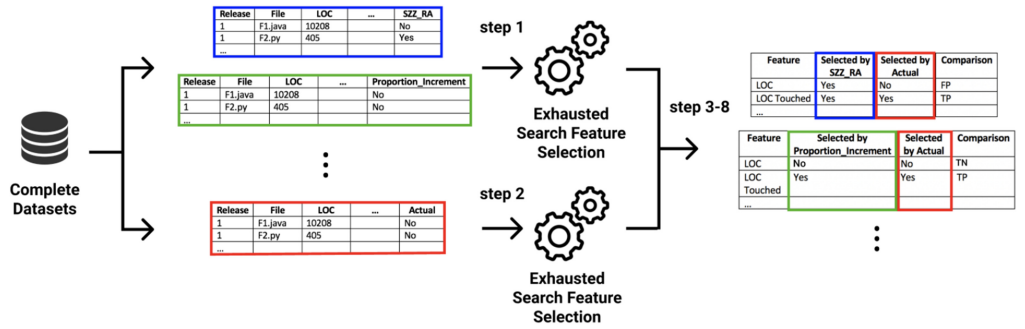


Fig. 9. The process to measure the accuracy of methods in leading to accurate feature selection.

Weka [42, 77]. This search technique performs an exhaustive search through the space of features subsets starting from the empty set of features. If two subsets have the same merit, which is also the best merit encountered, then the technique favors the smaller subset. We used CfsSubsetEval[9] for the evaluation function, which evaluates the worth of a subset of features by considering the individual predictive ability of each feature along with the degree of redundancy between them. Subsets of features that are highly correlated with the class while having low inter-correlation are preferred [27, 42].

(2) For each dataset P, for each version R, we set the class defectiveness according to the available actual/observed AVs, and we perform the Exhaustive Search Feature Selection on P_Actual_R using Weka and CfsSubsetEval as we did for the studied methods.

(3) For each P_M_R, we compare the features selected in P_M_R to the features selected in P_Actual_R.

Hypothesis testing. To test hypothesis H_{30}, we use the same statistical machinery used in RQ2 and RQ3. We perform Kruskal-Wallis tests followed by Dunn tests.

3.4.2 RQ4: Results. Figure 10 reports the distribution among datasets of the actual selection frequency of each feature. Since the frequency of selection varies across features, then it is important to select the correct set of features.

Regarding the comparison of the features selected on a dataset produced by a method (i.e., where the AVs are those retrieved by a method) versus the features selected by using the actual/observed dataset (i.e., where the AV are those provided be developers), Figure 11 reports the distribution of a certain method to retrieve AVs (x-axis, across versions and 76 Apache projects) of Precision, Recall, F1, MCC, and Kappa for selecting features. According to Figure 11, the proportion based methods have a higher accuracy (in all five metrics) compared to the studied SZZ methods. For example, according to Figure 11, the proportion-based methods are the only methods having a perfect median Precision and Recall.

Indeed, our Kruskall–Wallis and Dunn's tests reveal that hypothesis H_{30} (i.e., different methods have the same accuracy when selecting features) can be rejected. Therefore, we can claim that *retrieving AVs based on the defects' life cycle can lead to an overall, and on average, more accurate feature selection than the studied SZZ methods.*

Tables 8–10 show the Dunn's test results. Indeed, we observe that, in terms of $F1$, the proportion-based methods fall in better ranks than all the SZZ-based methods and the Simple method.

[9] https://weka.sourceforge.io/doc.dev/weka/attributeSelection/CfsSubsetEval.html.

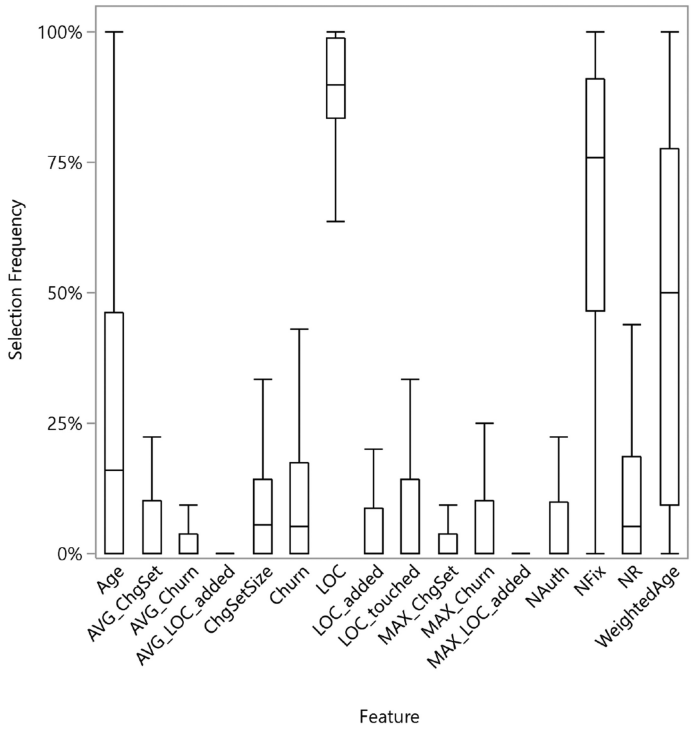


Fig. 10. Distribution among datasets of selection frequency of each feature.

However, we also note that the *Proportion Increment* yields significantly lower Recall and F1 values than the *Proportion Moving Window method*.

4 DISCUSSION

This section discuss our main results, the possible explanations for the results, implications, and guidelines for practitioners and researchers.

4.1 Main Results and Possible Explanations

The main implications of RQ1 is that most of the defects, of the 212 open-source projects from the Apache ecosystem, do not report AVs. However, according to Figure 3, the median project has most of the defects providing AVs. This means that in projects having a higher number of defects, there is a higher proportion of missing AVs compared to projects having a small number of defects.

The main result of RQ2 and RQ3 is that *all proportion -based methods have a higher precision and composite accuracy (F1, MCC, and Kappa) than all SZZ-based methods*. One of the possible reasons for the high accuracy achieved by the proportion-based methods is that P is substantially stable across projects (i.e., Proportion_ColdStart) and more stable within the same project (i.e., Proportion_Increment and Proportion_MovingWindow).

Figure 12 reports the distribution of values of IV, OV, FV, and P across defects of different projects. Table 11 reports the variation, in terms of standard deviation, of IV, OV, FV, and P when it is computed across different projects. According to both Figure 12 and Table 11, P is substantially stable across defects of different projects especially when compared to IV, OV, and FV.

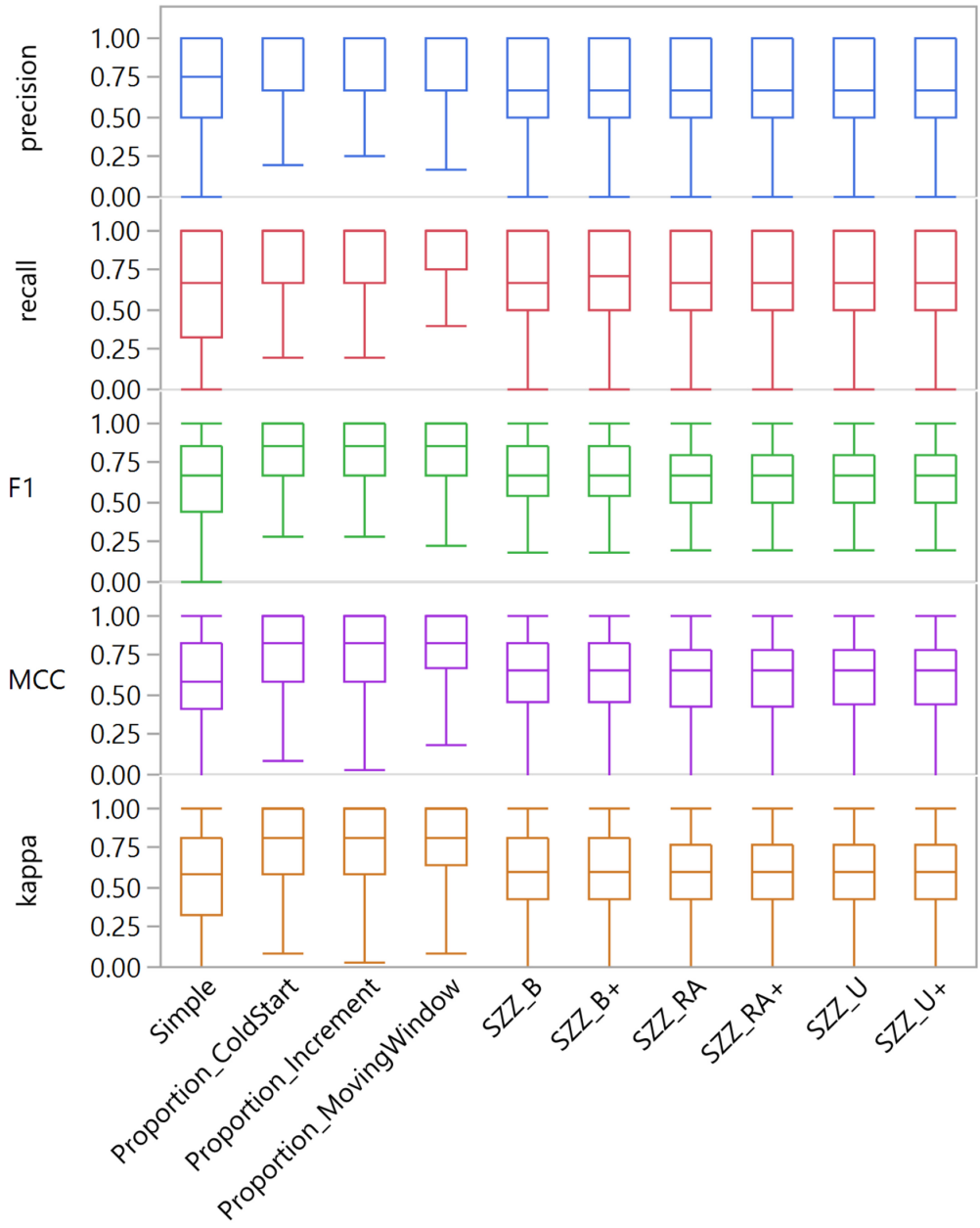


Fig. 11. Distribution, across versions and 76 Apache projects, of Precision, Recall, F1, MCC, and Kappa, of different methods in feature selection.

An additional relevant result of RQ3 is that *Proportion_MovingWindow method dominates all methods on all composite accuracy metrics (i.e., F1, MCC, and Kappa)*. This result is likely due to the fact that P is more stable within the same project than across projects. Figure 13 reports the distribution of standard deviation of IV, OV, FV, and P, across 76 Apache projects. Standard deviation is one way of measuring the amount of variation of a set of values [84]. A low standard

Table 8. Dunn's Test Results for the *Precision* Values of Hypothesis H_{30}

Method	Mean Precision	Dunn's Test Rank
Proportion Moving Window	0.84	1
Proportion Cold Start	0.83	1
Proportion Increment	0.82	1
Simple	0.74	2
SZZ_B+	0.70	3
SZZ_B	0.70	3
SZZ_U+	0.69	3
SZZ_U	0.69	3
SZZ_RA	0.69	3
SZZ_RA+	0.68	3

Table 9. Dunn's Test Results for the *Recall* Values of Hypothesis H_{30}

Method	Mean Recall	Dunn's Test Rank	Comments
Proportion Moving Window	0.84	1	
Proportion Cold Start	0.82	1	
Proportion Increment	0.81	1.5	Significantly lower than Proportion Moving Window
SZZ_B+	0.71	2	
SZZ_B	0.71	2	
SZZ_RA	0.70	2	
SZZ_U	0.70	2	
SZZ_RA+	0.70	2	
SZZ_U+	0.70	2	
Simple	0.61	3	

Table 10. Dunn's Test Results for the $F1$ Values of Hypothesis H_{30}

Method	Mean F1	Dunn's Test Rank	Comments
Proportion Moving Window	0.83	1	
Proportion Cold Start	0.81	1	
Proportion Increment	0.80	1.5	Significantly Lower than Proportion Moving Window
SZZ_B+	0.67	2	
SZZ_B	0.67	2	
SZZ_RA	0.66	2	
SZZ_U	0.66	2	
SZZ_U+	0.66	2	
SZZ_RA+	0.66	2	
Simple	0.64	2	

deviation (STDV) indicates that the values tend to be close to the mean and, hence, spread out over a narrow range. According to Figure 13, the STDV is much higher across projects than within the same project. Specifically, the median STDV of P computed within the same project is less than 2 (Figure 13) whereas the one across projects is about 5 (Table 11). In conclusion, Figure 3, Table 11, and Figure 13 (STDV <2) show that *the proportion of number of versions between their discovery and their fix is more stable within the same project than across different projects.*

The main results of RQ4 is that *the proportion-based methods have a higher accuracy (in all five metrics) than all SZZ methods.* Moreover, by observing Figure 11, we note that the accuracy of

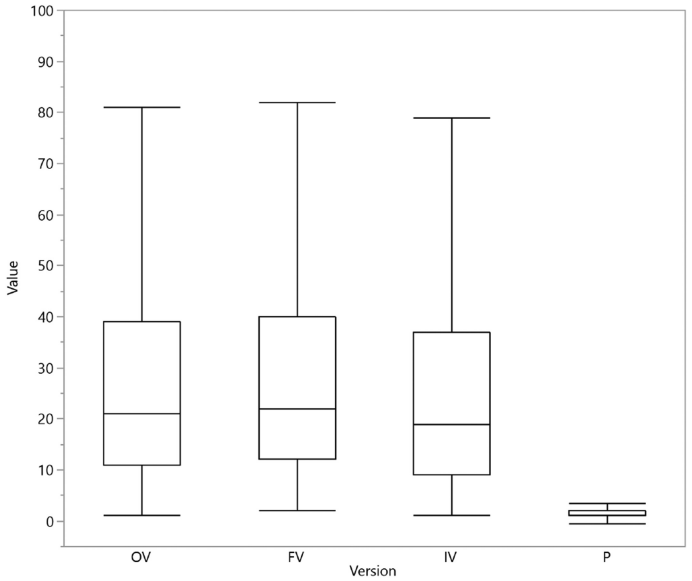


Fig. 12. Distribution of values of IV, OV, FV, and P across defects of 76 Apache projects.

Table 11. Variation, in Terms of Standard Deviation (STDV), of IV, OV, FV, and P Across Defects of 76 Apache Projects

Version	STDV
IV	38.36
OV	40.17
FV	41.85
P	5.43

SZZ methods can reach even a negative value of kappa. This means that a random selection of features is more accurate than a selection based on a dataset produced by an SZZ-based method.

RQ2, RQ3, and RQ4 share several results including that all proportion-based methods have a higher precision and composite accuracy than all SZZ-based methods. The major differences among RQ2 and RQ3 results is in that SZZ_B+ has a higher composite accuracy than Simple and any other SZZ method. One possible reason is that, in RQ3, a class can be affected by multiple defects and, therefore, methods which miss defects can still perform accurately (see discussion in Section 3.3.2). By comparing Figure 7 to Figure 5, we observe that all methods are more accurate for labeling classes (RQ3) than AVs (RQ2) on all accuracy metrics. Specifically, by comparing the median accuracy (across methods and datasets), we observe an increase in labeling classes over AVs of 13% in Precision, 5% in Recall, 16% in F1, 27% in MCC, and 39% in Kappa. It is interesting to note that the increase is higher in composite accuracy metrics than in atomic metrics. Again, we believe that one of the possible reasons is that, in RQ3, a class can be affected by multiple defects and, therefore, methods which miss defects can still perform accurately.

By comparing RQ4 to RQ2 and RQ3, we observe that there is less variation among accuracy of methods in RQ4 than in RQ2 or RQ3. In other words, the choice of the methods to retrieve AVs

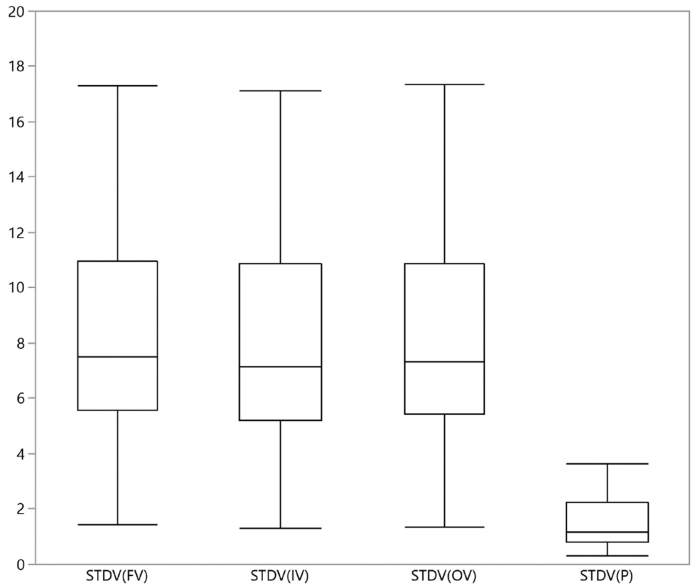


Fig. 13. Distribution of standard deviation of IV, OV, FV, and P, as computed within the same project, across 76 Apache projects.

has less impact on feature selection (RQ4) than on class labeling (RQ3). However, in RQ2 and RQ3, the proportion-based methods performed better than the other methods in four and three metrics, respectively, whereas in five metrics in RQ4. In other words, the superiority of the proportion-based methods is clearer in feature selection (RQ4) than in class labeling (RQ3). Another major difference between RQ4 to RQ2 and RQ3 is that the distribution of accuracy is much wider in RQ4 than in RQ2 or RQ3. For instance, when the methods are inaccurate they are extremely less accurate in RQ4 than in RQ2 or RQ3. In other words, a medium amount of inaccuracy in class labeling leaded to a big amount of inaccuracy in feature selection. For example, the lowest score of Proportion_MovingWindow in feature selection in Kappa, F1, Precision, and Recall is less than 0.25 in RQ4 but higher than 0.6 in RQ3. Similarly, the lowest scores of SZZ-based methods are even negative in case of Kappa for feature selection (RQ4) and higher than 0.2 in RQ3.

As different methods suggest different affected classes, then it could be possible to improve the accuracy of single methods in labeling defective classes by combining them through machine learning models. We tried several approaches, and our preliminary results were negative as they showed that feature selection—performed by using a machine learning model like random forest to merge the methods—identified Proportion_MovingWindow as the only important method. Thus, in the future, we plan to experiment with combining more methods, on different datasets, by using different feature selection techniques and machine learning models. Finally, someone could think that SZZ can be used more widely than Proportion as SZZ is decoupled from the development process used in a specific context and from possible changes in that process. However, the fact that Proportion_ColdStart outperforms SZZ suggests the opposite.

One of the possible reasons why SZZ methods have a higher Recall than proportion-based methods is that SZZ-based methods inflate the number of AVs as they produce a substantially higher number of defect-inducing changes when compared to the ones produced by the proportion-based methods. The results regarding F1 clearly show the cost of this inflation.

It could be that other window lengths outperform our current 1% window length. Moreover, it could even be that the best window length changes across projects or even within the same project. Thus, in future work, we plan to investigate how to dynamically define the moving window length across and within projects.

4.2 Implications

The main implications of RQ1 are that relying on the available AVs means neglecting most of the defects. Therefore, effort should be invested in retrieving AVs (hence, the importance of our work).

The main implications of RQ2 and RQ3 to practitioners is that SZZ should be used over proportional methods only in cases where higher Recall values are preferred over Precision, F1, MCC, and Kappa values. As for researchers, the results of RQ2 highlight exciting possibilities for future work in the area of defect introduction. For example, the framework proposed by Costa et al. [10] to evaluate SZZ implementations can be enhanced with the AVs retrieved by our proportion-based methods. The main implications of RQ4 results to practitioners and researchers is to prefer using the proportion-based methods over SZZ-based methods when performing feature selection. Specifically, practitioners can use our proportion-based methods and, more generally, our results in four different ways:

- Supporting decisions: the most obvious way to use proportion methods by practitioners is the one envisioned in RQ4; i.e., the practitioners can mine the dataset developed via proportion methods to understand what correlates with class defectiveness and make decisions according to this [2, 4, 15, 27, 39, 55, 57, 70, 76, 87]. For instance, in a previous paper [18], the third author reported on a custom Web-based tool called the Measurement Reporting Tool used in a software company called Keymind to support decisions about software releases based on the predicted number of defects that a version would have. Possible decisions include the level of compliance of the code to the architecture and the number of requirements to implement in that version. If the datasets to mine would have been developed via SZZ, then this would lead us to the use of the wrong features, incurring wrong decisions regarding versions, thus incurring more defects, and, hence, more failures. This would ultimately lead to an economic loss of the company.
- Supporting testing: an additional way to use proportion methods is to mine the dataset developed via proportion methods for predicting which class is prone to be defective. During testing, the developers can focus on classes that are prone to be defective and, hence, avoiding more defects given a limited amount of effort resources. However, if the datasets to mine would have been developed via SZZ, then this would lead us to the use of the wrong features. These wrong features would, in turn, lead to inaccurate prediction of defect prone classes, which would, in turn, as explained before, ultimately lead to an economic loss of the company.
- Supporting Jira: a further way to use our proportion-based methods is to be installed in an issue tracking system like Jira and, when the user would have to provide the AV information, the tool could suggest the potential AVs as retrieved by the Proportion_MovingWindow. The tool could also provide a warning message asking the user to confirm the submitted AV if it is very different from the suggested AV.
- Debugging: finally, practitioners can use the AVs retrieved by Proportion_MovingWindow for debugging activities. Given that a new defect is reported and awaiting for a fix, the related retrieved IV could be used to understand the context in which the defect has been introduced, e.g., the developers could think of the features introduced in that IV as the

potential source of the defect. Helping developers to narrow down to the version when a defect was introduced would considerably help to find the mistake to be fixed.

The overall main implication to researchers is twofold:

(1) the need to revisit the studies that have used SZZ methods as the oracle upon which the studies have based their conclusions [23, 34, 36, 37].
(2) the need to provide more accurate methods as no method to label affected versions or defective classes is perfect. The Kappa in Figures 5 and 7 is far from 1.0 and, therefore, future studies are needed.

Finally, we would like to propose some guidelines for researchers and practitioners. Researchers should refrain from using SZZ for creating oracles. Researchers should create the oracles by using the available AV and, when missing, they should use the Proportion_MovingWindow method. Since no proportion method is perfect, and likely no method will ever be perfect, researchers should prefer mining projects with a high proportion of available and consistent AV. Practitioners should carefully report the AV in all defect reports; they could be aided by the proportion-based method in case they believe providing AV is complex.

5 THREATS TO VALIDITY

In this section, we report the threats to validity of our study. The section is organized by threat type, i.e., Conclusion, Internal, Construct, and External.

5.1 Conclusion

Conclusion validity concerns issues that affect the ability to draw accurate conclusions regarding the observed relationships between the independent and dependent variables [78].

We tested all hypotheses with non-parametric tests (e.g., Kruskal–Wallis), which are prone to type-2 error, i.e., not rejecting a false hypothesis. We have been able to reject the hypotheses in most of the cases; therefore, the likelihood of a type-2 error is low. Moreover, the alternative would have been using parametric tests (e.g., ANOVA) that are prone to type-1 error, i.e., rejecting a true hypothesis, which, in our context, is less desirable than type-2 error. Also, we acknowledge that our proposed methods (i.e., independent variables) do not represent an exhaustive list of methods that could have been implemented (for example, one could use machine learning to optimize the proportions used in the ColdStart method). However, our proposed methods are a simple and effective baseline to start with (as shown by our obtained results).

5.2 Internal

Internal validity is concerned with the influences that can affect the independent variables with respect to causality [78]. A threat to *internal validity* is the lack of ground truth for class defectiveness, which could have been underestimated in our measurements. In other words, the AVs provided by developers might be inaccurate due to human error. Nevertheless, we would argue that this is a common threat in most of empirical research in the area of software engineering [35].

5.3 Construct

Construct validity is concerned with the degree to which our measurements indeed reflect what we claim to measure [78].

In our study, we compare our proposed proportion methods with the SZZ-based methods. We are aware that the output of SZZ are defect-introducing changes and not affected versions. For example, although SZZ may output three distinct defect-introducing changes (which we may interpret as three distinct affected versions), we do not investigate the dependency between these defect-introducing changes. For instance, a defect may only be present when all the three defect-introducing changes are present. Therefore, a version that contains only one of the defect-introducing changes may not be, in actuality, an affected version. Nevertheless, our assumptions are aligned with prior work, which has considered every potential defect-introducing change as indeed defect-introducing [10] and, therefore, can be interpreted as incurring an affected version.

Moreover, we use Precision, Recall, F1-Score, Matthews Correlation Coefficient, and Cohen's Kappa to measure the accuracy for labeling defectiveness in RQ2 and RQ3. Although we do not use the AUC metric, which is a threshold-free metric [73], our methods do not output probabilities. Therefore, our evaluations are not impacted by threshold choices.

5.4 External

External validity is concerned with the extent to which the research elements (subjects, artifacts, etc.) are representative of actual elements [78].

This study used a large set of datasets and, hence, could be deemed of high generalization compared to similar studies. Of course, our results cannot be generalized by projects that would significantly differ from the settings used in this present study.

Finally, in order to promote reproducible research, all datasets, results, and scripts for this article are available in our replication package.[10]

6 CONCLUSION

In this article, we first measured the AV availability and consistency in open-source projects, and then evaluated a new method for retrieving AVs, i.e., the origin of a defect, which is based on the idea that defects have a stable life cycle in terms of proportion of number of versions required to discover and to fix the defect. Our results regarding 212 open-source projects from the Apache ecosystem, featuring a total of about 125,000 defects, show that the AVs cannot be used in the majority (51%) of defect reports. Therefore, it is important to develop automated methods to retrieve AVs. Results related to 76 open-source projects from the Apache ecosystem, featuring a total of about 6,250,000 classes that are affected by 60,000 defects and spread over 4,000 versions and 760,000 commits, show that our proposed methods are, on average, more accurate when compared to previously proposed and state-of-the-art SZZ-based methods, for retrieving AVs. Our results suggest that our proposed methods are also better than SZZ-based methods for labeling classes as defective and for developing defects repositories to perform feature selection. In conclusion, our proposed methods are a valid automated alternative to SZZ for estimating the origin of a defect and, hence, for building defects repository for defect prediction endeavors.

Future studies include:

— **Analyzing other defect-introducing commits in SZZ methods**. In our research, we selected the earliest possible defect-introducing commit returned by SZZ to be the IV for a defect. Future work will focus on how selecting later defect-introducing commits affects the accuracy in labeling classes in versions as defective or not.

[10] https://gitlab.com/Bvandehei/affectedversions.

— **Analyzing the role of reporting affect versions to developers**. In our study, we only analyzed whether AVs were available and consistent. Future work will focus on why and how developers report AVs—How do developers determine AVs? Do developers find reporting AVs important?
— **Replication in context of JIT**. JIT prediction models, where the predicted variable is the defectiveness of a commit, have become sufficiently robust that they are now incorporated into the development cycle of some companies [47]. Therefore, it is important to investigate the accuracy of proportion based methods in the context of JIT models.
— **Finer combination of proportion-based and SZZ-based methods**. In this work, we have combined SZZ and proportion-based methods by simply tagging a version as defective if it came after the defect report creation and not tagged by SZZ. More finer combinations are possible, including the use of ML; i.e., the dataset to evaluate and use ML models can be created by ML models.
— **Use a finer P**. In this work, we simply used the proportion of versions to find and to fix a defect to determine P, which is then used to label AVs and classes. However, there exists room for improvement in calculating P. For example, P can be improved using Linear Regression. In addition to the version information, the number of days can also be used.

REFERENCES

[1] Aalok Ahluwalia, Davide Falessi, and Massimiliano Di Penta. 2019. Snoring: A noise in defect prediction datasets. In *Proceedings of the 16th International Conference on Mining Software Repositories*.

[2] Muhammad Asaduzzaman, Michael C. Bullock, Chanchal K. Roy, and Kevin A. Schneider. 2012. Bug introducing changes: A case study with android. In *Proceedings of the 9th IEEE Working Conference of Mining Software Repositories, MSR 2012, June 2–3, 2012, Zurich, Switzerland*, Michele Lanza, Massimiliano Di Penta, and Tao Xie (Eds.). IEEE Computer Society, 116–119. DOI: https://doi.org/10.1109/MSR.2012.6224267

[3] Victor R. Basili, Lionel C. Briand, and Walcélio L. Melo. 1996. A validation of object-oriented design metrics as quality indicators. *IEEE Transactions on Software Engineering* 22, 10 (1996), 751–761.

[4] Mario Luca Bernardi, Gerardo Canfora, Giuseppe A. Di Lucca, Massimiliano Di Penta, and Damiano Distante. 2012. Do developers introduce bugs when they do not communicate? The case of Eclipse and Mozilla. In *Proceedings of the 16th European Conference on Software Maintenance and Reengineering, CSMR 2012, Szeged, Hungary, March 27–30, 2012*, Tom Mens, Anthony Cleve, and Rudolf Ferenc (Eds.). IEEE Computer Society, 139–148. DOI: https://doi.org/10.1109/CSMR.2012.24

[5] Christian Bird, Adrian Bachmann, Eirik Aune, John Duffy, Abraham Bernstein, Vladimir Filkov, and Premkumar Devanbu. 2009. Fair and balanced?: Bias in bug-fix datasets. In *Proceedings of the the 7th Joint Meeting of the European Software Engineering Conference and the ACM SIGSOFT Symposium on The Foundations of Software Engineering (ESEC/FSE'09)*. ACM, New York, NY, 121–130. DOI: https://doi.org/10.1145/1595696.1595716

[6] Markus Borg, Oscar Svensson, Kristian Berg, and Daniel Hansson. 2019. SZZ unleashed: An open implementation of the SZZ algorithm—Featuring example usage in a study of just-in-time bug prediction for the Jenkins project. *CoRR* abs/1903.01742 (2019). arxiv:1903.01742 http://arxiv.org/abs/1903.01742

[7] Jianfeng Chen, Joymallya Chakraborty, Philip Clark, Kevin Haverlock, Snehit Cherian, and Tim Menzies. 2019. Predicting breakdowns in cloud services (with SPIKE). In *Proceedings of the 2019 27th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. 916–924.

[8] Tse-Hsun Chen, Meiyappan Nagappan, Emad Shihab, and Ahmed E. Hassan. 2014. An empirical study of dormant bugs. In *Proceedings of the 11th Working Conference on Mining Software Repositories—MSR 2014*. DOI: https://doi.org/10.1145/2597073.2597108

[9] Jacob Cohen. 1960. A coefficient of agreement for nominal scales. *Educational and Psychological Measurement* 20, 1 (1960), 37–46.

[10] Daniel Alencar Da Costa, Shane McIntosh, Weiyi Shang, Uirá Kulesza, Roberta Coelho, and Ahmed E. Hassan. 2017. A framework for evaluating the results of the SZZ approach for identifying bug-introducing changes. *IEEE Transactions on Software Engineering* 43, 7 (2017), 641–657.

[11] Daniel Alencar da Costa, Shane McIntosh, Weiyi Shang, Uirá Kulesza, Roberta Coelho, and Ahmed E. Hassan. 2017. A framework for evaluating the results of the SZZ approach for identifying bug-introducing changes. *IEEE Transactions on Software Engineering* 43, 7 (2017), 641–657. DOI: https://doi.org/10.1109/TSE.2016.2616306

[12] Marco D'Ambros, Michele Lanza, and Romain Robbes. 2012. Evaluating defect prediction approaches: A benchmark and an extensive comparison. *Empirical Software Engineering* 17, 4–5 (Aug. 2012), 531–577. DOI: https://doi.org/10.1007/s10664-011-9173-9

[13] Olive Jean Dunn. 1964. Multiple comparisons using rank sums. *Technometrics* 6, 3 (1964), 241–252.

[14] Jon Eyolfson, Lin Tan, and Patrick Lam. 2011. Do time of day and developer experience affect commit bugginess? In *Proceedings of the 8th Working Conference on Mining Software Repositories*. ACM, 153–162.

[15] Jon Eyolfson, Lin Tan, and Patrick Lam. 2011. Do time of day and developer experience affect commit bugginess. In *Proceedings of the 8th International Working Conference on Mining Software Repositories, MSR 2011 (Co-located with ICSE), Waikiki, Honolulu, HI, May 21–28, 2011, Proceedings*, Arie van Deursen, Tao Xie, and Thomas Zimmermann (Eds.). ACM, 153–162. DOI: https://doi.org/10.1145/1985441.1985464

[16] Davide Falessi and Max Jason Moede. 2018. Facilitating feasibility analysis: The pilot defects prediction dataset maker. In *Proceedings of the 4th ACM SIGSOFT International Workshop on Software Analytics, SWAN@ESEC/SIGSOFT FSE 2018, Lake Buena Vista, FL, November 5, 2018*. 15–18.

[17] Davide Falessi, Barbara Russo, and Kathleen Mullen. 2017. What if I had no smells? In *Proceedings of the 11th ACM/IEEE International Symposium on Empirical Software Engineering and Measurement (ESEM'17)*. IEEE Press, Piscataway, NJ, 78–84. DOI: https://doi.org/10.1109/ESEM.2017.14

[18] Davide Falessi, Michele A. Shaw, and Kathleen Mullen. 2014. Achieving and maintaining CMMI maturity level 5 in a small organization. *IEEE Software* 31, 5 (2014), 80–86. DOI: https://doi.org/10.1109/MS.2014.17

[19] Davide Falessi, Wyatt Smith, and Alexander Serebrenik. 2017. STRESS: A semi-automated, fully replicabile approach for project selection. In *Proceedings of the 11th ACM/IEEE International Symposium on Empirical Software Engineering and Measurement (ESEM'17)*. IEEE Press, Piscataway, NJ, 151–156.

[20] Yuanrui Fan, Xin Xia, Daniel Alencar Da Costa, David Lo, Ahmed E Hassan, and Shanping Li. 2019. The impact of changes mislabeled by SZZ on just-in-time defect prediction. *IEEE Transactions on Software Engineering* (2019).

[21] Wei Fu and Tim Menzies. 2017. Revisiting unsupervised learning for defect prediction. In *Proceedings of the 2017 11th Joint Meeting on Foundations of Software Engineering*. 72–83.

[22] Wei Fu, Tim Menzies, and Xipeng Shen. 2016. Tuning for software analytics: Is it really necessary?*Information & Software Technology* 76 (2016), 135–146.

[23] Takafumi Fukushima, Yasutaka Kamei, Shane McIntosh, Kazuhiro Yamashita, and Naoyasu Ubayashi. 2014. An empirical study of just-in-time defect prediction using cross-project models. In *Proceedings of the 11th Working Conference on Mining Software Repositories*. 172–181.

[24] Kehan Gao, Taghi M. Khoshgoftaar, Huanjing Wang, and Naeem Seliya. 2011. Choosing software metrics for defect prediction: An investigation on feature selection techniques. *Software: Practice and Experience* 41, 5 (2011), 579–606.

[25] Baljinder Ghotra, Shane McIntosh, and Ahmed E. Hassan. 2017. A large-scale study of the impact of feature selection techniques on defect classification models. In *Proceedings of the 2017 IEEE/ACM 14th International Conference on Mining Software Repositories (MSR)*. IEEE, 146–157.

[26] Georgios Gousios and Diomidis Spinellis. 2014. Conducting quantitative software engineering studies with Alitheia core. *Empirical Software Engineering* 19, 4 (2014), 885–925.

[27] M. A. Hall. 1998. *Correlation-based Feature Subset Selection for Machine Learning*. Ph.D. Dissertation. University of Waikato, Hamilton, New Zealand.

[28] Maurice Howard Halstead et al. 1977. *Elements of Software Science*. Vol. 7. Elsevier New York.

[29] Steffen Herbold. 2017. Comments on ScottKnottESD in response to "an empirical comparison of model validation techniques for defect prediction models". *IEEE Transactions on Software Engineering* 43, 11 (2017), 1091–1094. DOI: https://doi.org/10.1109/TSE.2017.2748129

[30] Kim Herzig, Sascha Just, and Andreas Zeller. 2013. It's not a bug, it's a feature: How misclassification impacts bug prediction. In *Proceedings of the 2013 International Conference on Software Engineering (ICSE'13)*. IEEE Press, Piscataway, NJ, 392–401. http://dl.acm.org/citation.cfm?id=2486788.2486840

[31] Sture Holm. 1979. A simple sequentially rejective multiple test procedure. *Scandinavian Journal of Statistics* (1979), 65–70.

[32] Tian Jiang, Lin Tan, and Sunghun Kim. 2013. Personalized defect prediction. In *Proceedings of the 2013 28th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 279–289.

[33] Yasutaka Kamei, Takafumi Fukushima, Shane McIntosh, Kazuhiro Yamashita, Naoyasu Ubayashi, and Ahmed E. Hassan. 2016. Studying just-in-time defect prediction using cross-project models. *Empirical Software Engineering* 21, 5 (2016), 2072–2106.

[34] Yasutaka Kamei, Shinsuke Matsumoto, Akito Monden, Ken-ichi Matsumoto, Bram Adams, and Ahmed E. Hassan. 2010. Revisiting common bug prediction findings using effort-aware models. In *Proceedings of the 26th IEEE International Conference on Software Maintenance (ICSM 2010), September 12–18, 2010, Timisoara, Romania*. IEEE Computer Society, 1–10. DOI: https://doi.org/10.1109/ICSM.2010.5609530

[35] Yasutaka Kamei and Emad Shihab. 2016. Defect prediction: Accomplishments and future challenges. In *Proceedings of the 2016 IEEE 23rd International Conference on Software Analysis, Evolution, and Reengineering (SANER)*, Vol. 5. IEEE, 33–45.

[36] Yasutaka Kamei, Emad Shihab, Bram Adams, Ahmed E. Hassan, Audris Mockus, Anand Sinha, and Naoyasu Ubayashi. 2012. A large-scale empirical study of just-in-time quality assurance. *IEEE Transactions on Software Engineering* 39, 6 (2012), 757–773.

[37] Sunghun Kim, E. James Whitehead Jr., and Yi Zhang. 2008. Classifying software changes: Clean or buggy?*IEEE Transactions on Software Engineering* 34, 2 (2008), 181–196.

[38] Sunghun Kim, Hongyu Zhang, Rongxin Wu, and Liang Gong. 2011. Dealing with noise in defect prediction. In *Proceedings of the 33rd International Conference on Software Engineering (ICSE'11)*. ACM, New York, NY, 481–490.

[39] Sunghun Kim, Thomas Zimmermann, E. James Whitehead Jr., and Andreas Zeller. 2007. Predicting faults from cached history. In *Proceedings of the 29th International Conference on Software Engineering (ICSE 2007), Minneapolis, MN, May 20–26, 2007*. IEEE Computer Society, 489–498. DOI: https://doi.org/10.1109/ICSE.2007.66

[40] Sunghun Kim, Thomas Zimmermann, Kai Pan, and E. James Jr. Whitehead. 2006. Automatic identification of bug-introducing changes. In *Proceedings of the 21st IEEE/ACM International Conference on Automated Software Engineering (ASE'06)*. IEEE Computer Society, Washington, DC, 81–90. DOI: https://doi.org/10.1109/ASE.2006.23

[41] Pavneet Singh Kochhar, Yuan Tian, and David Lo. 2014. Potential biases in bug localization: Do they matter? In *Proceedings of the 29th ACM/IEEE International Conference on Automated Software Engineering (ASE'14)*. ACM, New York, NY, 803–814.

[42] Masanari Kondo, Cor-Paul Bezemer, Yasutaka Kamei, Ahmed E. Hassan, and Osamu Mizuno. 2019. The impact of feature reduction techniques on defect prediction models. *Empirical Software Engineering* 24, 4 (2019), 1925–1963.

[43] William H. Kruskal and W. Allen Wallis. 1952. Use of ranks in one-criterion variance analysis. *Journal of the American Statistical Association* 47, 260 (1952), 583–621.

[44] Janaki T. Madhavan and E. James Whitehead Jr. 2007. Predicting buggy changes inside an integrated development environment. In *Proceedings of the 2007 OOPSLA Workshop on Eclipse Technology Exchange*. 36–40.

[45] Markus Borg, Oscar Svensson, Kristian Berg, and Daniel Hansson. 2019. SZZ Unleashed: An open implementation of the SZZ algorithm - Featuring example usage in a study of just-in-time bug prediction for the jenkins project. In *Proc. of the Workshop on Machine Learning Techniques for Software Quality Evolution (MaLTeSQuE'19)*. 7–12.

[46] Thomas J. McCabe. 1976. A complexity measure. *IEEE Transactions on Software Engineering*4 (1976), 308–320.

[47] Shane McIntosh and Yasutaka Kamei. 2018. Are fix-inducing changes a moving target? A longitudinal case study of just-in-time defect prediction. *IEEE Transactions on Software Engineering* 44, 5 (2018), 412–428.

[48] J. Sayyad Shirabad and T. J. Menzies. 2005. The PROMISE repository of software engineering databases. School of Information Technology and Engineering, University of Ottawa, Canada. http://promise.site.uottawa.ca/SERepository.

[49] Nuthan Munaiah, Steven Kroh, Craig Cabrey, and Meiyappan Nagappan. 2017. Curating github for engineered software projects. *Empirical Software Engineering* 22, 6 (Dec. 2017), 3219–3253. DOI: https://doi.org/10.1007/s10664-017-9512-6

[50] Meiyappan Nagappan, Thomas Zimmermann, and Christian Bird. 2013. Diversity in software engineering research. In *Proceedings of the 2013 9th Joint Meeting on Foundations of Software Engineering (ESEC/FSE 2013)*. ACM, New York, NY, 466–476.

[51] Jaechang Nam, Wei Fu, Sunghun Kim, Tim Menzies, and Lin Tan. 2017. Heterogeneous defect prediction. *IEEE Transactions on Software Engineering* 44, 9 (2017), 874–896.

[52] Jaechang Nam and Sunghun Kim. 2015. Clami: Defect prediction on unlabeled datasets (t). In *Proceedings of the 2015 30th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 452–463.

[53] Edmilson Campos Neto, Daniel Alencar Da Costa, and Uirá Kulesza. 2018. The impact of refactoring changes on the SZZ algorithm: An empirical study. In *Proceedings of the 25th International Conference on Software Analysis, Evolution and Reengineering, SANER 2018, Campobasso, Italy, March 20–23, 2018*. 380–390. DOI: https://doi.org/10.1109/SANER.2018.8330225

[54] Annibale Panichella, Carol V. Alexandru, Sebastiano Panichella, Alberto Bacchelli, and Harald C. Gall. 2016. A search-based training algorithm for cost-aware defect prediction. In *Proceedings of the Genetic and Evolutionary Computation Conference 2016*. 1077–1084.

[55] Foyzur Rahman, Christian Bird, and Premkumar T. Devanbu. 2010. Clones: What is that smell? In *Proceedings of the 7th International Working Conference on Mining Software Repositories, MSR 2010 (Co-located with ICSE), Cape Town, South Africa, May 2–3, 2010*, Jim Whitehead and Thomas Zimmermann (Eds.). IEEE Computer Society, 72–81. DOI: https://doi.org/10.1109/MSR.2010.5463343

[56] Foyzur Rahman and Premkumar Devanbu. 2013. How, and why, process metrics are better. In *Proceedings of the 2013 35th International Conference on Software Engineering (ICSE)*. IEEE, 432–441.

[57] Foyzur Rahman and Premkumar T. Devanbu. 2011. Ownership, experience and defects: A fine-grained study of authorship. In *Proceedings of the 33rd International Conference on Software Engineering, ICSE 2011, Waikiki, Honolulu,*

HI, May 21–28, 2011, Richard N. Taylor, Harald C. Gall, and Nenad Medvidovic (Eds.). ACM, 491–500. DOI: https://doi.org/10.1145/1985793.1985860

[58] Foyzur Rahman, Daryl Posnett, Israel Herraiz, and Premkumar Devanbu. 2013. Sample Size vs. bias in defect prediction. In *Proceedings of the 2013 9th Joint Meeting on Foundations of Software Engineering (ESEC/FSE 2013)*. ACM, New York, NY, 147–157.

[59] Izabela A. Rodenhuis-Zybert, Jan Wilschut, and Jolanda M. Smit. 2010. Dengue virus life cycle: Viral and host factors modulating infectivity. *Cellular and Molecular Life Sciences* 67, 16 (2010), 2773–2786.

[60] Gema Rodríguez-Pérez, Gregorio Robles, and Jesús M. González-Barahona. 2018. Reproducibility and credibility in empirical software engineering: A case study based on a systematic literature review of the use of the SZZ algorithm. *Information & Software Technology* 99 (2018), 164–176.

[61] Gema Rodríguez-Pérez, Gregorio Robles, Alexander Serebrenik, Andy Zaidman, Daniel M. Germán, and Jesus M. Gonzalez-Barahona. [n.d.]. How bugs are born: A model to identify how bugs are introduced in software components. *Empirical Software Engineering* ([n.d.]), 1–47.

[62] Gema Rodríguez-Pérez, Andy Zaidman, Alexander Serebrenik, Gregorio Robles, and Jesús M. González-Barahona. 2018. What if a bug has a different origin?: Making sense of bugs without an explicit bug introducing change. In *Proceedings of the 12th ACM/IEEE International Symposium on Empirical Software Engineering and Measurement (ESEM'18)*. ACM, New York, NY, Article 52, 4 pages. DOI: https://doi.org/10.1145/3239235.3267436

[63] Daniel Rozenberg, Ivan Beschastnikh, Fabian Kosmale, Valerie Poser, Heiko Becker, Marc Palyart, and Gail C. Murphy. 2016. Comparing repositories visually with RepoGrams. In *Proceedings of the 13th International Conference on Mining Software Repositories, MSR 2016, Austin, TX, May 14–22, 2016*. 109–120.

[64] Giuseppe Scanniello, Carmine Gravino, Andrian Marcus, and Tim Menzies. 2013. Class level fault prediction using software clustering. In *Proceedings of the 2013 28th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 640–645.

[65] Samuel Sanford Shapiro and Martin B. Wilk. 1965. An analysis of variance test for normality (complete samples). *Biometrika* 52, 3/4 (1965), 591–611.

[66] Martin Shepperd, Qinbao Song, Zhongbin Sun, and Carolyn Mair. 2013. Data quality: Some comments on the NASA software defect datasets. *IEEE Transactions on Software Engineering* 39, 9 (Sept. 2013), 1208–1215.

[67] Martin Shepperd, Qinbao Song, Zhongbin Sun, and Carolyn Mair. 2013. Data quality: Some comments on the NASA software defect datasets. *IEEE Transactions on Software Engineering* 39, 9 (2013), 1208–1215.

[68] Shivkumar Shivaji, E. James Whitehead Jr., Ram Akella, and Sunghun Kim. 2009. Reducing features to improve bug prediction. In *Proceedings of the 2009 IEEE/ACM International Conference on Automated Software Engineering*. IEEE, 600–604.

[69] Jacek Śliwerski, Thomas Zimmermann, and Andreas Zeller. 2005. When do changes induce fixes? In *Proceedings of the 2005 International Workshop on Mining Software Repositories (MSR'05)*. ACM, New York, NY, 1–5. DOI: https://doi.org/10.1145/1082983.1083147

[70] Jacek Sliwerski, Thomas Zimmermann, and Andreas Zeller. 2005. When do changes induce fixes? *ACM SIGSOFT Software Engineering Notes* 30, 4 (2005), 1–5. DOI: https://doi.org/10.1145/1082983.1083147

[71] Chakkrit Tantithamthavorn, Shane McIntosh, Ahmed E. Hassan, Akinori Ihara, and Kenichi Matsumoto. 2015. The impact of mislabelling on the performance and interpretation of defect prediction models. In *Proceedings of the 37th International Conference on Software Engineering—Volume 1 (ICSE'15)*. IEEE Press, Piscataway, NJ, 812–823. http://dl.acm.org/citation.cfm?id=2818754.2818852

[72] Chakkrit Tantithamthavorn, Shane McIntosh, Ahmed E. Hassan, and Kenichi Matsumoto. 2016. Automated parameter optimization of classification techniques for defect prediction models. In *Proceedings of the 38th International Conference on Software Engineering, ICSE 2016, Austin, TX, May 14–22, 2016*. 321–332.

[73] Chakkrit Tantithamthavorn, Shane McIntosh, Ahmed E. Hassan, and Kenichi Matsumoto. 2016. An empirical comparison of model validation techniques for defect prediction models. *IEEE Transactions on Software Engineering* 43, 1 (2016), 1–18.

[74] Chakkrit Tantithamthavorn, Shane McIntosh, Ahmed E. Hassan, and Kenichi Matsumoto. 2018. The impact of automated parameter optimization on defect prediction models. *CoRR* abs/1801.10270 (2018). arxiv:1801.10270 http://arxiv.org/abs/1801.10270

[75] Kapil Vaswani and Abhik Roychoudhury. 2010. Approach for root causing regression bugs. U.S. Patent App. 12/469,850.

[76] Cathrin Weiß, Rahul Premraj, Thomas Zimmermann, and Andreas Zeller. 2007. How long will it take to fix this bug? In *Proceedings of the 4th International Workshop on Mining Software Repositories, MSR 2007 (ICSE Workshop), Minneapolis, MN, May 19–20, 2007*. IEEE Computer Society, 1. DOI: https://doi.org/10.1109/MSR.2007.13

[77] Ian H. Witten, Eibe Frank, and Mark A. Hall. 2011. *Data Mining: Practical Machine Learning Tools and Techniques* (3rd ed.). Morgan Kaufmann Publishers Inc., San Francisco, CA.

[78] Claes Wohlin, Per Runeson, Martin Hst, Magnus C. Ohlsson, Bjrn Regnell, and Anders Wessln. 2012. *Experimentation in Software Engineering*. Springer Publishing Company, Incorporated.

[79] Xinli Yang, David Lo, Xin Xia, Yun Zhang, and Jianling Sun. 2015. Deep learning for just-in-time defect prediction. In *Proceedings of the 2015 IEEE International Conference on Software Quality, Reliability and Security*. IEEE, 17–26.

[80] Xiaoxing Yang, Ke Tang, and Xin Yao. 2012. A learning-to-rank algorithm for constructing defect prediction models. In *International Conference on Intelligent Data Engineering and Automated Learning*. Springer, 167–175.

[81] Xiaoxing Yang, Ke Tang, and Xin Yao. 2014. A learning-to-rank approach to software defect prediction. *IEEE Transactions on Reliability* 64, 1 (2014), 234–246.

[82] Yibiao Yang, Yuming Zhou, Jinping Liu, Yangyang Zhao, Hongmin Lu, Lei Xu, Baowen Xu, and Hareton Leung. 2016. Effort-aware just-in-time defect prediction: Simple unsupervised models could be better than supervised models. In *Proceedings of the 2016 24th ACM SIGSOFT International Symposium on Foundations of Software Engineering*. 157–168.

[83] Suraj Yatish, Jirayus Jiarpakdee, Patanamon Thongtanunam, and Chakkrit Tantithamthavorn. 2019. Mining software defects: Should we consider affected releases? (2019), 654–665. DOI: https://doi.org/10.1109/ICSE.2019.00075

[84] Arthur B. Yeh. 2005. Fundamentals of probability and statistics for engineers. *Technometrics* 47, 2 (2005), 239. DOI: https://doi.org/10.1198/tech.2005.s266

[85] Xiao Yu, Kwabena Ebo Bennin, Jin Liu, Jacky Wai Keung, Xiaofei Yin, and Zhou Xu. 2019. An empirical study of learning to rank techniques for effort-aware defect prediction. In *Proceedings of the 2019 IEEE 26th International Conference on Software Analysis, Evolution and Reengineering (SANER)*. IEEE, 298–309.

[86] Xiao Yu, Jin Liu, Jacky Wai Keung, Qing Li, Kwabena Ebo Bennin, Zhou Xu, Junping Wang, and Xiaohui Cui. 2019. Improving ranking-oriented defect prediction using a cost-sensitive ranking SVM. *IEEE Transactions on Reliability* (2019).

[87] Thomas Zimmermann, Rahul Premraj, and Andreas Zeller. 2007. Predicting defects for eclipse. In *Proceedings of the 3rd International Workshop on Predictor Models in Software Engineering (PROMISE'07)*. IEEE Computer Society, Washington, DC, 9–.

Received March 2020; revised October 2020; accepted November 2020